



# Protocoles et Services de base d'Internet

Master 2ITIC

Dr. Maïssa Mbaye

[maissa.mbaye@ugb.edu.sn](mailto:maissa.mbaye@ugb.edu.sn)

Université Gaston Berger de St-Louis/Sénégal

# Partie 3 : Couche Transport

## **PARTIE 3 : COUCHE TRANSPORT**

### **Plan**

- **Généralités**
- **UDP**
- **TCP**

## PARTIE 3 : COUCHE TRANSPORT

### Plan

- **Généralités**
- UDP
- TCP

# Motivations de la couche transport

- Limitations d'IP
  - Mode de fonctionnement non connecté
    - Aucune assurance que le destinataire est disponible pour recevoir les paquets
  - Non fiable et non acquitté
    - Aucune assurance que le destinataire a reçu les paquets de manière convenable

# Fonctions de la couche transport

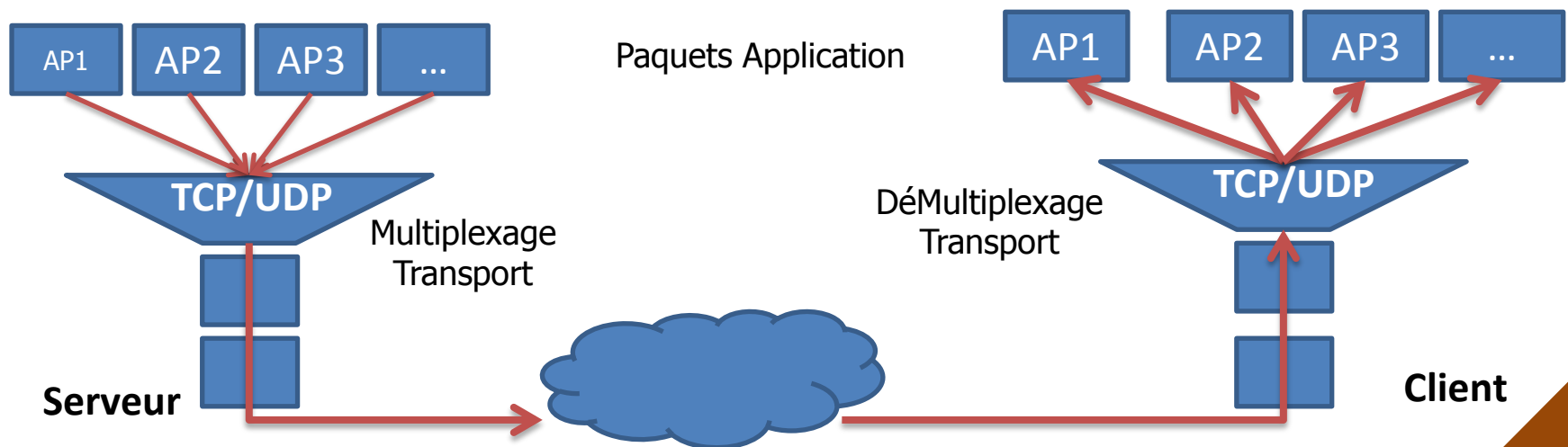
- Adressage au niveau processus
  - Adressage pour différencier les différents logiciels partageant les mêmes couches basses
- Multiplexage et Démultiplexage
- Segmentation Packaging et réassemblage
- Etablissement gestion et terminaison de connexions
- Acquittement et retransmission de paquets
- Contrôle de flux

# Protocoles de Transports : UDP et TCP

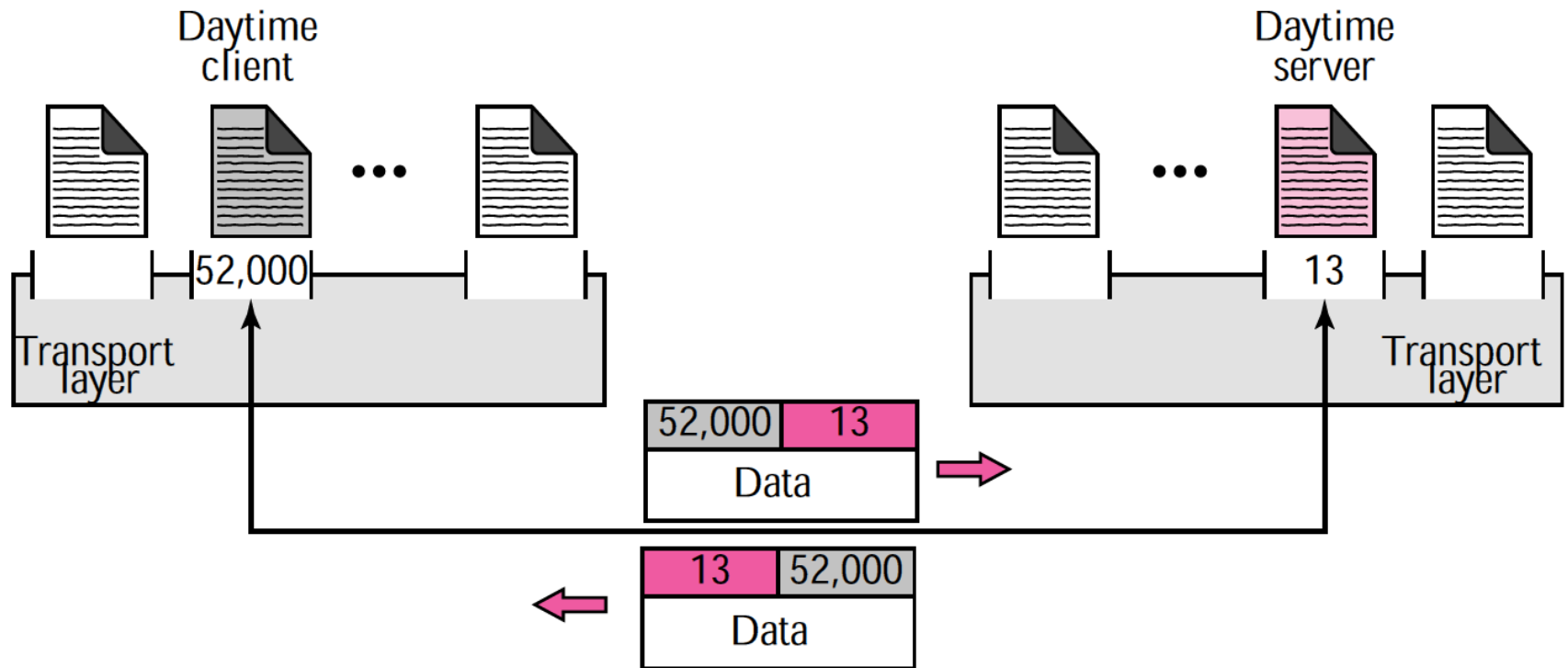
- TCP (Transmission Control Protocol)
  - Protocole de transport orienté connexion, fiable, acquitté...
  - Conçu pour les applications avec un besoin de fiabilité, ou d'acquiescement, ...
  - Exemples : HTTP, WWW, SMTP,
- UDP (User Datagram Protocol)
  - Un Protocole de transport simple offrant un adressage de niveau transport (comme TCP)
  - Conçu pour des applications sans exigences fortes en terme de fiabilité ou que la réémission de paquets perdus n'est pas désirée
  - Exemple : vidéo streaming et multimédia

# Multiplexage / Démultiplexage niveau transport

- Multiplexage
  - Mécanisme permettant de faire transiter les données de plusieurs processus applications sur une même couche IP
- Démultiplexage
  - Mécanisme permettant de séparer les données de plusieurs processus applications qui arrivent sur une même couche IP



# Multiplexage / Démultiplexage niveau transport



## Adressage de niveau transport : Socket et Ports

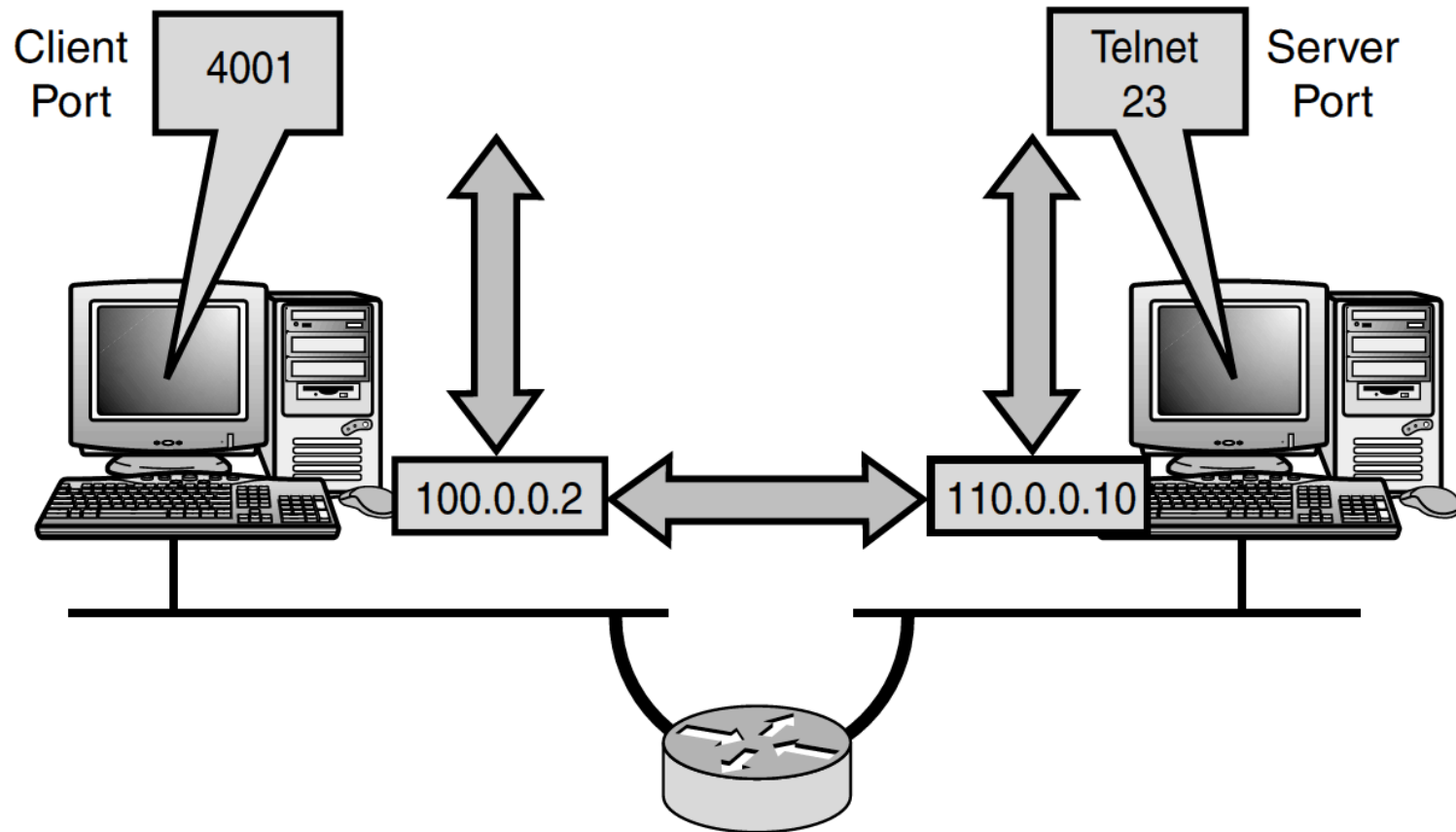
- Adressage supplémentaire à l'adressage réseau (IP)
  - Permet le fonctionnement simultané de plusieurs applications partageant la même adresse IP
  - Adressage virtuelle de logiciels permettant l'indentification des différents processus dans le processus de multiplexage/démultiplexage
- Adressage basée sur les ports et les sockets

# Adresses de ports

- Adresse Port ou Numéro de port
  - Identifie de manière unique avec le champs *Protocol* d'IP le processus applicatif source et destinataire au niveau transport (UDP/TCP)
  - Nombre de 16bits (théoriquement compris entre 0 et 65535)
- Trois différents usages
  - [0 ➔ 1023 ] : ports **réservés** d'applications bien connues
    - Allocation par l'IANA par des standards (RFC). Exemple 80 WWW
    - Aussi appelés ports système
  - [1024 ➔ 49151 ] : ports **alloués** d'applications commerciales
    - Dépôts d'entreprises privées à l'IANA pour s'assurer de ne pas avoir de conflits
  - [49152 ➔ 65535 ]:
    - Ports libres utilisables pour n'importe quelle application privée
    - Adapté pour des applications utilisateurs



# Adresses de ports



# Adresses de ports réservés

- Assignations géré par l'IANA
  - Liste des ports alloués disponibles dans un RFC dans le passé (dernier RFC 1700 *Assigned Number Standard* publié en 1994)
  - Depuis 2002 la RFC 2002 est déclarée obsolète
    - Remplacé par le lien  
<http://www.iana.org/assignments/port-numbers>

## Quelques ports réservés des applications bien connues

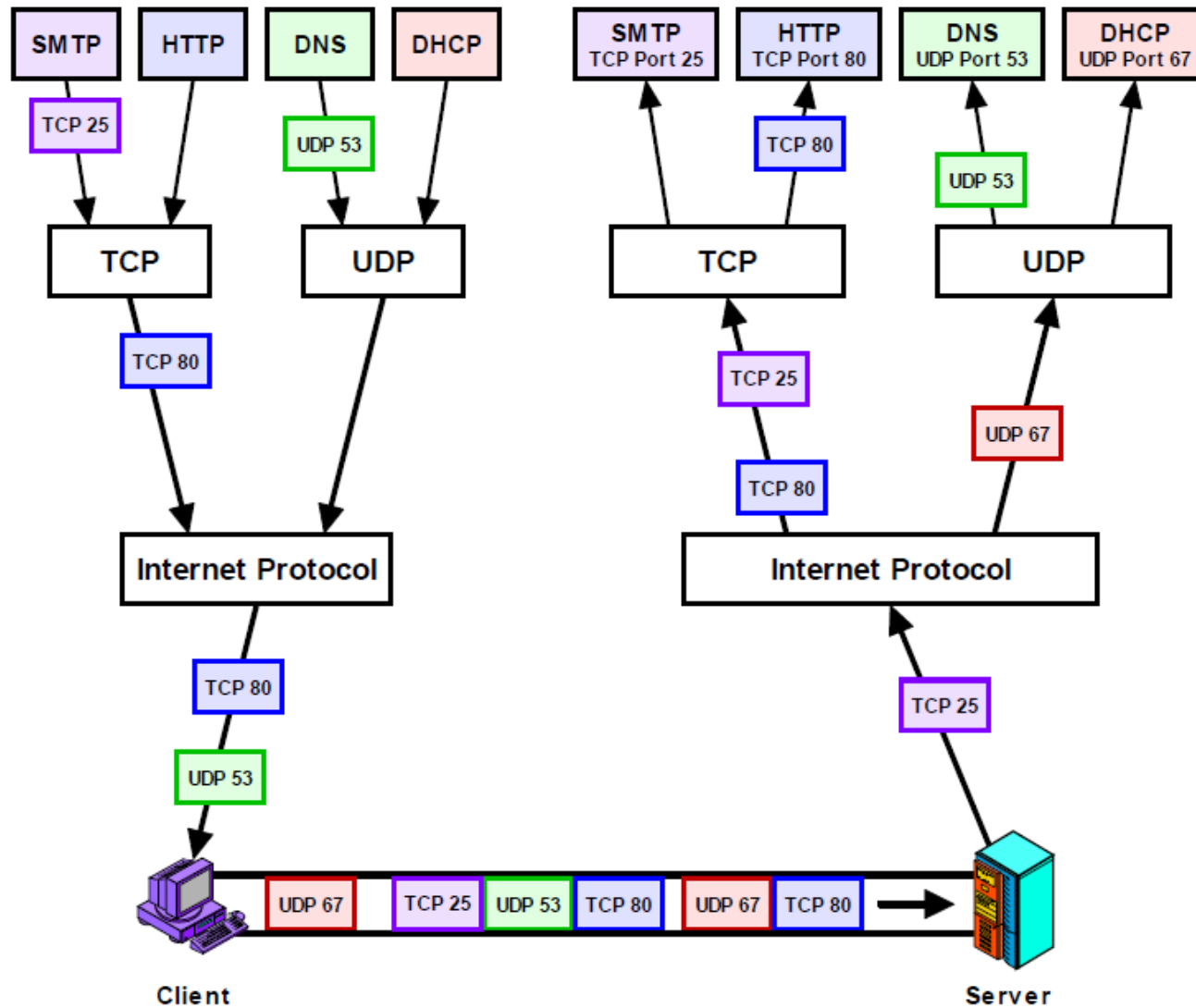
| Port # | TCP / UDP | Keyword  | Protocol Abbreviation | Application or Protocol Name / Comments                           |
|--------|-----------|----------|-----------------------|-------------------------------------------------------------------|
| 7      | TCP + UDP | echo     | —                     | Echo Protocol                                                     |
| 9      | TCP + UDP | discard  | —                     | Discard Protocol                                                  |
| 11     | TCP + UDP | systat   | —                     | Active Users Protocol                                             |
| 13     | TCP + UDP | daytime  | —                     | Daytime Protocol                                                  |
| 17     | TCP + UDP | qotd     | QOTD                  | Quote Of The Day Protocol                                         |
| 19     | TCP + UDP | chargen  | —                     | Character Generator Protocol                                      |
| 20     | TCP       | ftp-data | FTP (data)            | File Transfer Protocol (default data port)                        |
| 21     | TCP       | ftp      | FTP (control)         | File Transfer Protocol (control / commands)                       |
| 23     | TCP       | telnet   | —                     | Telnet Protocol                                                   |
| 25     | TCP       | smtp     | SMTP                  | Simple Mail Transfer Protocol                                     |
| 37     | TCP + UDP | time     | —                     | Time Protocol                                                     |
| 43     | TCP       | nicname  | —                     | Whois Protocol (also called "Nickname")                           |
| 53     | TCP + UDP | domain   | DNS                   | Domain Name Server (Domain Name System)                           |
| 67     | UDP       | bootps   | BOOTP / DHCP          | Bootstrap Protocol / Dynamic Host Configuration Protocol (Server) |

# Quelques numéros de ports alloués

| Port # | TCP / UDP | Keyword | Protocol Abbreviation | Application or Protocol Name / Comments   |
|--------|-----------|---------|-----------------------|-------------------------------------------|
| 1512   | TCP + UDP | wins    | WINS                  | Microsoft Windows Internet Naming Service |
| 1701   | UDP       | l2tp    | L2TP                  | Layer Two Tunneling Protocol              |

| Port #      | TCP / UDP | Keyword | Protocol Abbreviation | Application or Protocol Name / Comments |
|-------------|-----------|---------|-----------------------|-----------------------------------------|
| 1723        | TCP       | pptp    | PPTP                  | Point-To-Point Tunneling Protocol       |
| 2049        | TCP + UDP | nfs     | NFS                   | Network File System                     |
| 6000 - 6063 | TCP       | x11     | X11                   | X Window System                         |

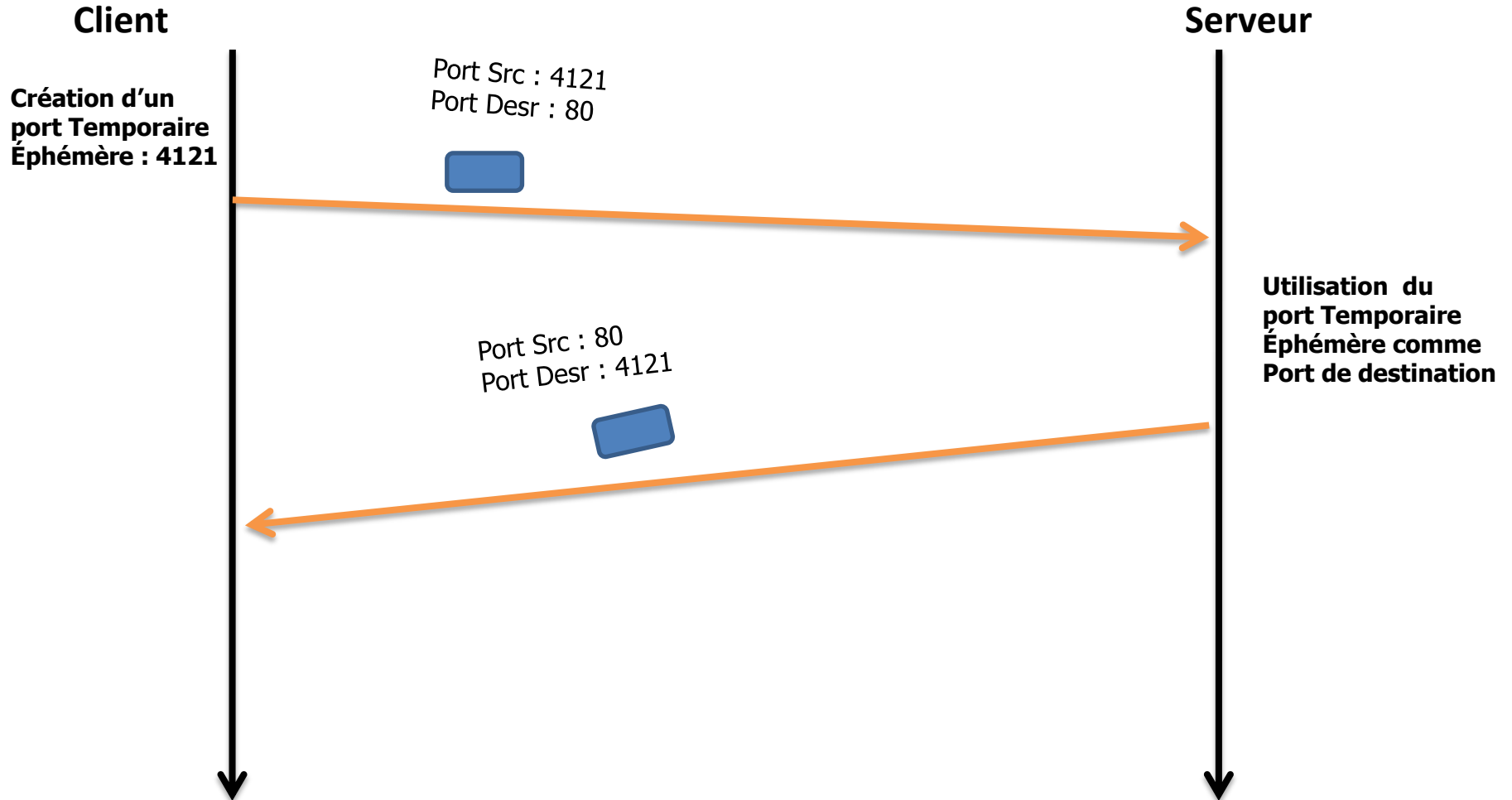
# Fonctionnement des ports transport



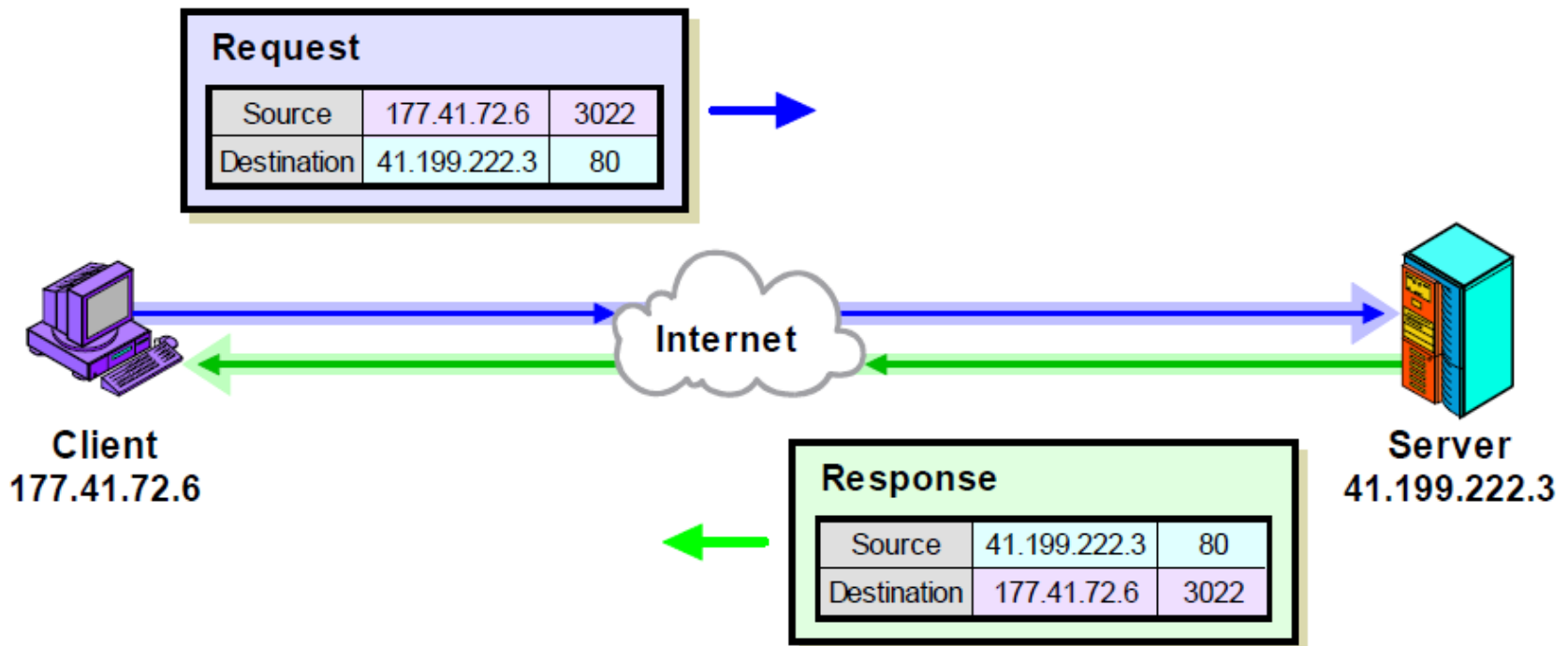
# Ports clients éphémères

- TCP et UDP fonctionnent en mode Client/Serveur
  - Le client initie une requête vers le serveur
  - Conséquences :
    - le client doit connaître à l'avance le numéro port du serveur
      - ➔ le numéro de port des processus serveurs doivent être universellement connus
      - ➔ Numéro de ports réservés
    - Le client utilise un port non réservé éphémère pour son adresse port
      - Pour une réutilisation maximale de la plage de ports non réservés
      - Pour permettre la présence d'un serveur et son client sur la même machine
      - Le serveur trouve l'adresse port du client dans le champs *Source Port* de la requête

# Ports clients éphémères



# Fonctionnement des ports éphémères

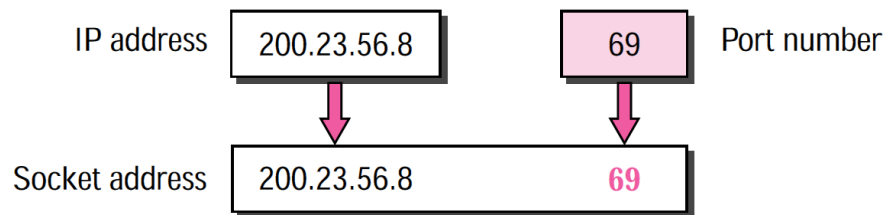


# Plage des ports éphémères

- Dépendent des implémentations de TCP/IP
  - Classiquement les ports éphémères BSD sont dans l'intervalle **[1024→4999]** soit 3976 ports clients
  - Théoriquement il n y a qu'une limite
    - Ne pas utiliser un port réservé ou alloué
- Allocation effectuée par le système d'exploitation
  - Nécessité que la pile TCP monitore l'activité des ports éphémères alloués

# Identification de processus : Sockets

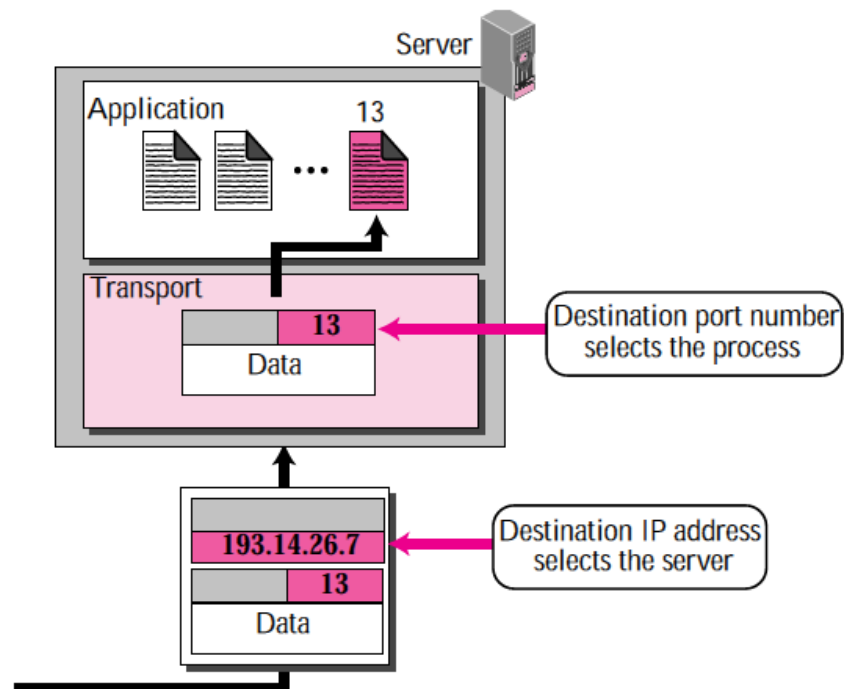
- Un socket identifie un processus d'application
  - Combinaison : Adresse IP de l'interface + Numéro de port
  - Notations des sockets:
    - <Protocol><Adresse IP>:<Numéro de Port>
    - <Protocol><Nom Hôte>:<Numéro de Port>
  - Exemple :
    - tcp:41.199.222.3:80 ou http://www.unsiteinteressant.com:8080



- Pairs de sockets
  - Socket serveur + socket client
  - Identifie une connexion (TCP), deux processus échangeant des données (UDP)

# Identification de processus : Sockets

- L'adresse IP détermine la machine alors que le numéro de port détermine le processus



# API(application programming interface) Sockets

- Ensembles de fonctions génériques permettant le traitement des communications à travers des sockets
  - Première version proposé dans Unix BSD 4.2
  - Aucune standardisation mais est devenue de facto une standard industrielle
  - Mise à jour de l'API Socket pour IPv6 dans le RFC 3493 et 3542
- Trois services pour les applications
  - Service Socket en mode stream → TCP
    - Exemple : FTP
  - Service Socket en mode datagramme → UDP
    - Exemple : NFS
  - Service Socket en mode Raw → Accès direct aux couches basses (IP, ICMP)
    - Souvent utilisé pour tester les nouveaux protocoles → **ping** utilise ce mode

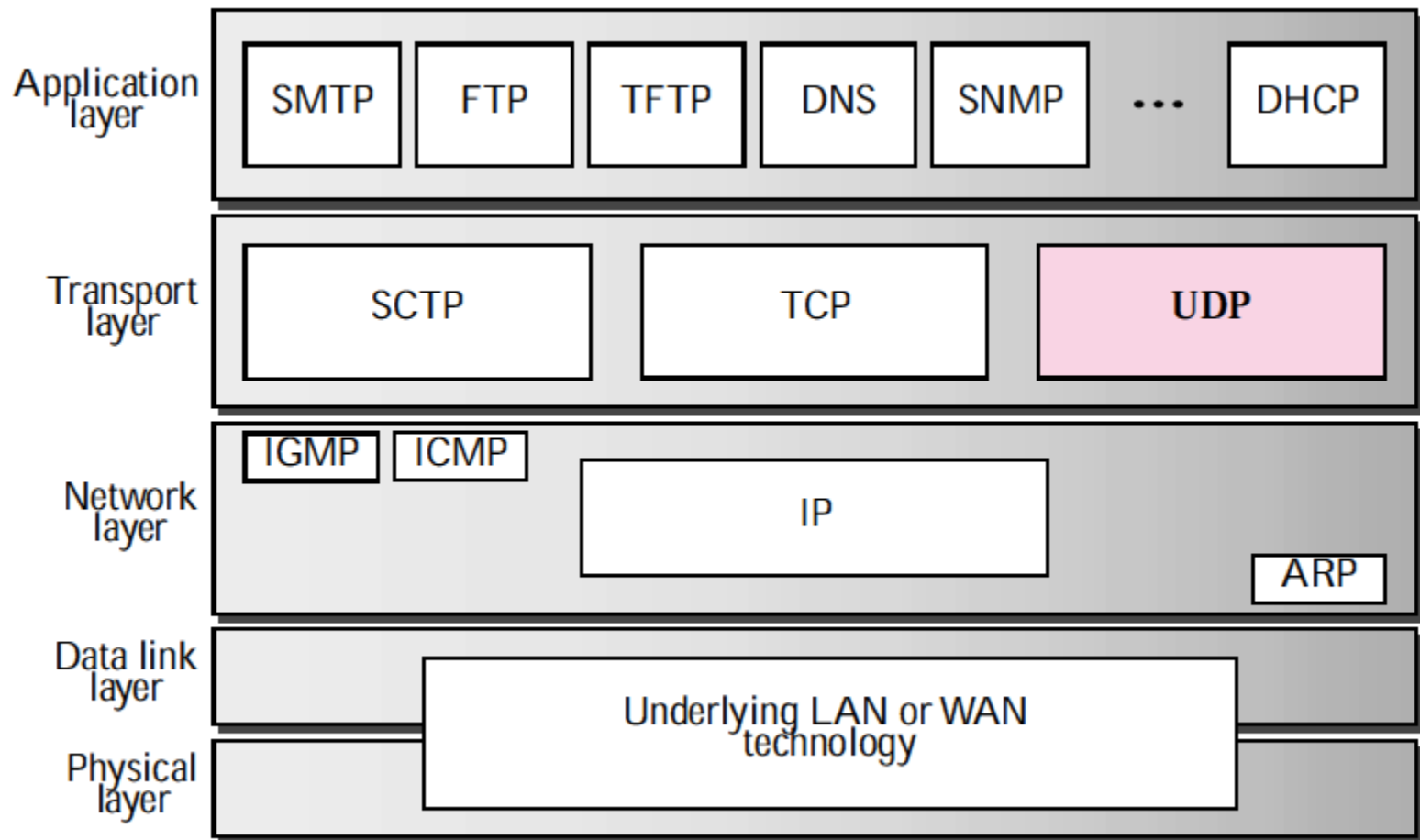
## PARTIE 3 : COUCHE TRANSPORT

### Plan

- Généralités
- **UDP**
- TCP



# UDP dans l'architecture TCP/IP



# UDP : Présentation

- User Datagram Protocol
  - Protocole de niveau transport
- Objectifs
  - Fournir un service transport qui n'offre pas les garanties de TCP
    - Pour les applications qui n'en ont pas besoin
    - Ou les applications qui ne fonctionnent pas bien avec les traitement TCP
- Caractéristiques de base
  - Simplicité
    - Définit dans la RFC 768 (1980) qui ne fait que trois pages et jamais modifiée
  - Rapidité
    - UDP ne fait pratiquement aucun traitements sur les données applicatives

# Opérations UDP

- Transport de données des couches hautes
  - Les applications peuvent envoyer au logiciel UDP pour un transport de bout en bout
- Encapsulation des données
  - Les données des couches supérieures sont encapsulées dans la partie données des messages UDP
  - UDP ajoute un en-tête port source et port destination
- Transfère des paquets à la couche IP
  - Les données UDP sont transmises à la couche IP pour une encapsulation de niveau réseau

# Ce qu'UDP ne fait pas

- Etablissement de connexion
  - Une connexion n'est pas établie avant l'envoi des données
- Acquittement des données
  - UDP ne s'assure pas que les données sont bien reçues
- Détection de pertes et retransmission
  - UDP ne détecte pas des données perdues ou n'a aucun mécanisme de retransmission

# Ce qu'UDP ne fait pas

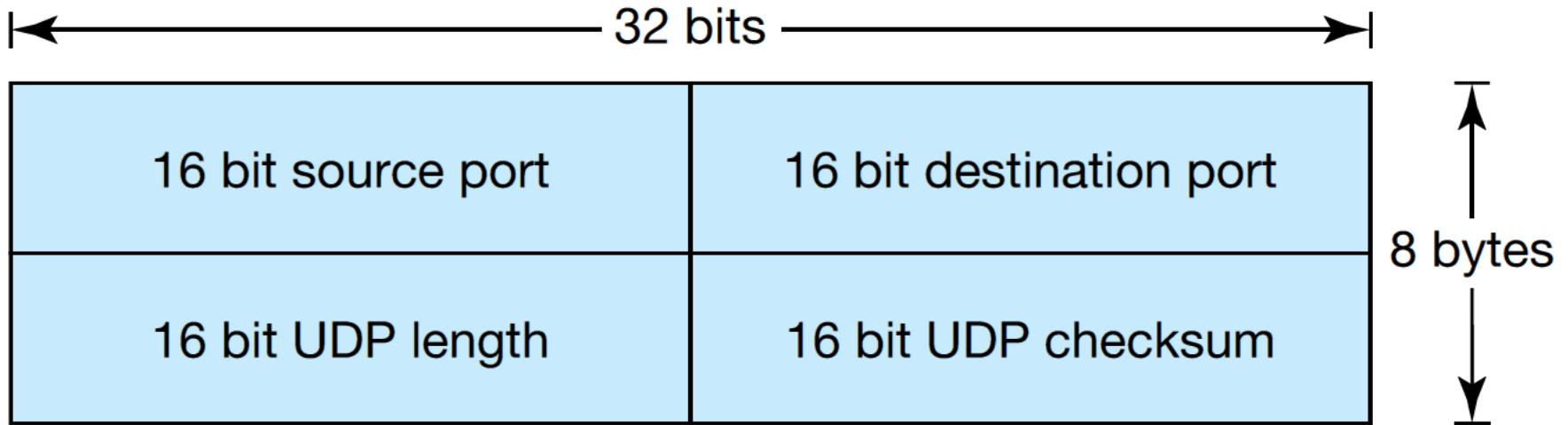
- Détection de l'ordre d'arrivée des données
  - UDP ne s'assure pas que les données sont reçues dans la même séquence qu'au moment de l'envoi
- Mécanisme de gestion de congestion et de control de flux
  - UDP ne contrôle pas les flux (gestion des débits d'émission)

**UDP = IP + Port Adressing**

**→ UDP est un wrapper protocol**

# Format de message UDP

- Entête UDP sur seulement 8 octets
  - 16bit pour le numéro de port source et 16bits pour la destination
  - Length(16bits) : longueur du paquet UDP (entête inclus)
  - Checksum contrôle d'erreurs sur 16 bits



# Exercice

Soit l'entête UDP suivant au format  
Héxadécimal

**CB84000D001C001C**

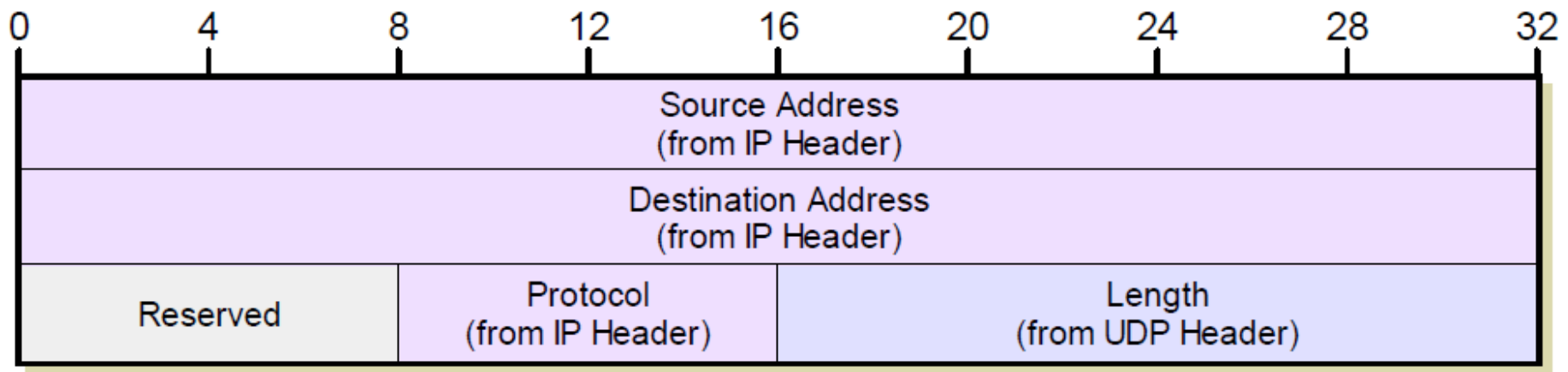
- a) Quel est le numéro de port source ?
- b) Quel est le numéro de port destinataire
- c) Quelle est la longueur totale du datagramme
- d) Quelle est la direction du paquet : client ==> Serveur  
ou l'inverse ?
- e) Quel est le processus client

# Format de message UDP

- **Port Source**
  - Numéro de port du client (source) (éphémère du client) sur 2octets.
- **Port Destination**
  - Numéro de port serveur (serveur) réservé/alloué sur 2octets.
- **Longueur**
  - Longueur du datagramme entête + données
- **Cheacksum**
  - Champs optionnel de checksum sur l'ensemble du paquet et un champs **speudo-entête**
- **Données**
  - Données des couches hautes encapsulées

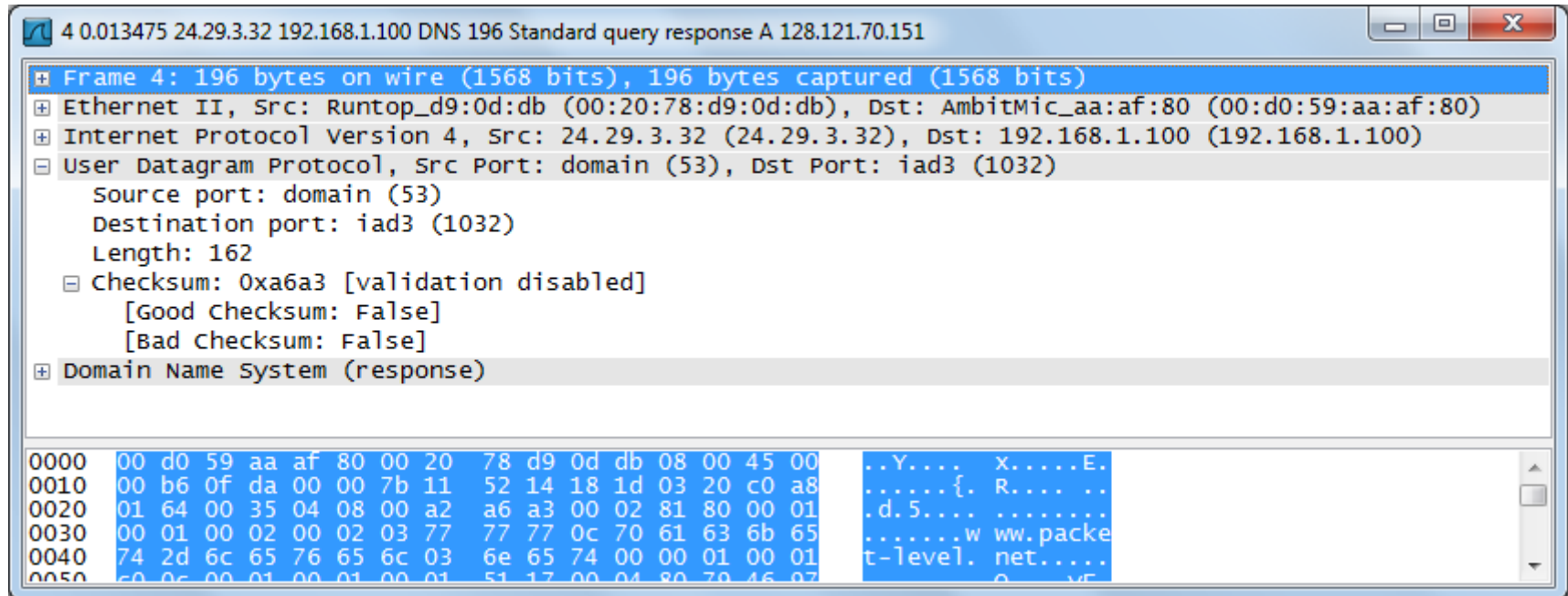
# Champs Cheacksum et Speudo header

- Cheacksum
  - Mécanisme de protection contre les erreurs
  - Calculé sur le datagramme original et un speudo



Speudo header UDP

# Exemple de datagramme UDP



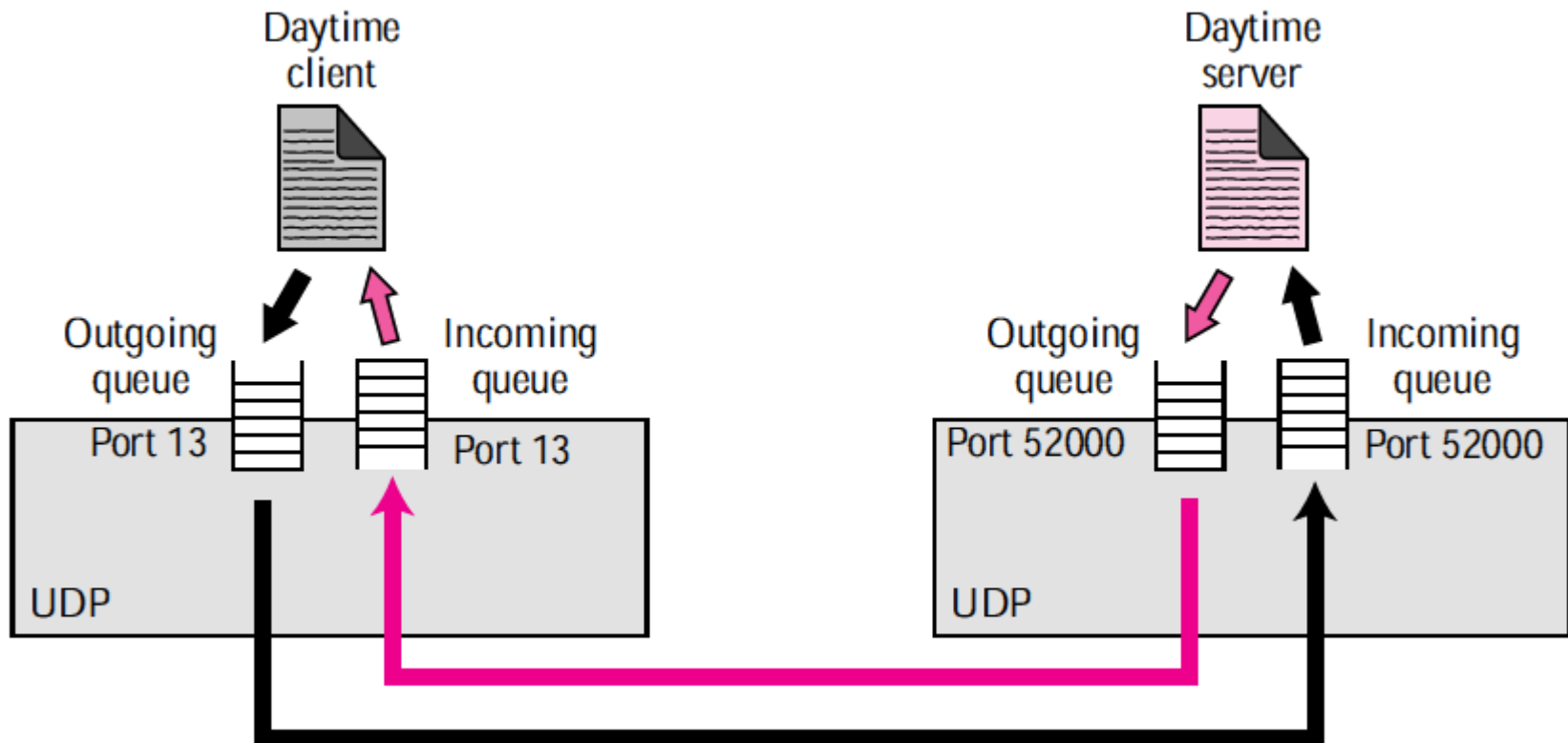
# Applications communes d'UDP

| <i>Port</i> | <i>Protocol</i> | <i>Description</i>                            |
|-------------|-----------------|-----------------------------------------------|
| 7           | Echo            | Echoes a received datagram back to the sender |
| 9           | Discard         | Discards any datagram that is received        |
| 11          | Users           | Active users                                  |
| 13          | Daytime         | Returns the date and the time                 |
| 17          | Quote           | Returns a quote of the day                    |
| 19          | Chargen         | Returns a string of characters                |
| 53          | Nameserver      | Domain Name Service                           |
| 67          | Boothps         | Server port to download bootstrap information |
| 68          | Bootpc          | Client port to download bootstrap information |
| 69          | TFTP            | Trivial File Transfer Protocol                |
| 111         | RPC             | Remote Procedure Call                         |
| 123         | NTP             | Network Time Protocol                         |
| 161         | SNMP            | Simple Network Management Protocol            |
| 162         | SNMP            | Simple Network Management Protocol (trap)     |

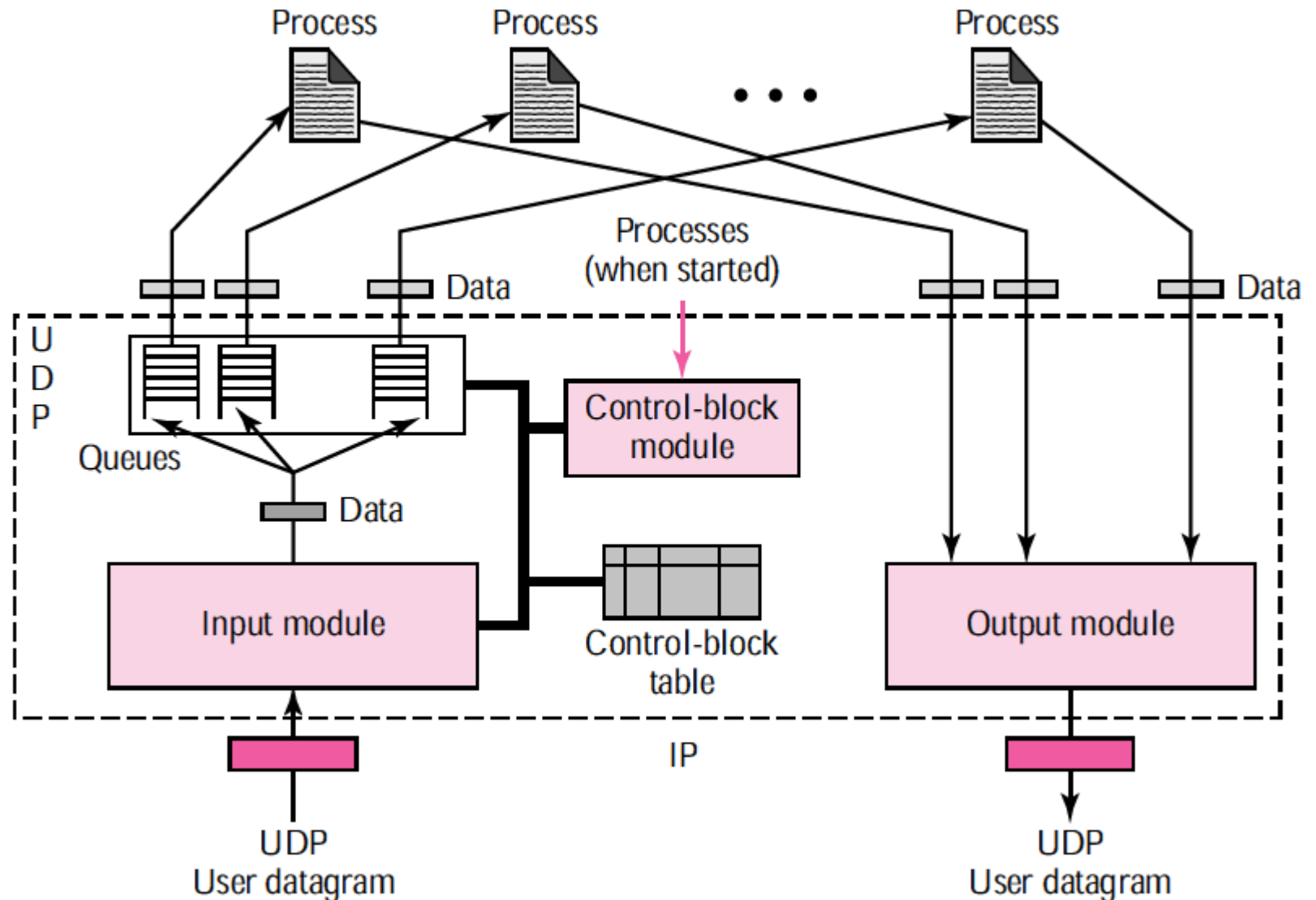
# Cas d'utilisations d'UDP

- Quand la performance prime sur la complétude
  - Application multimédias : la perte de quelques données est moins grave que la rupture de la communication
- Echange de données de petites tailles
  - Quand les données peuvent tenir dans un datagramme il n'y a pas besoin de contrôle de l'ordre d'arrivée des paquets
- Application multicast
  - TCP ne supporte que de l'unicast
- Application prenant en charge la gestion de pertes
  - Mécanisme de fiabilité dupliqué avec TCP

# Files d'attente UDP



# Architecture d'implémentation d'UDP



## PARTIE 2 : COUCHE APPLICATION

### Plan

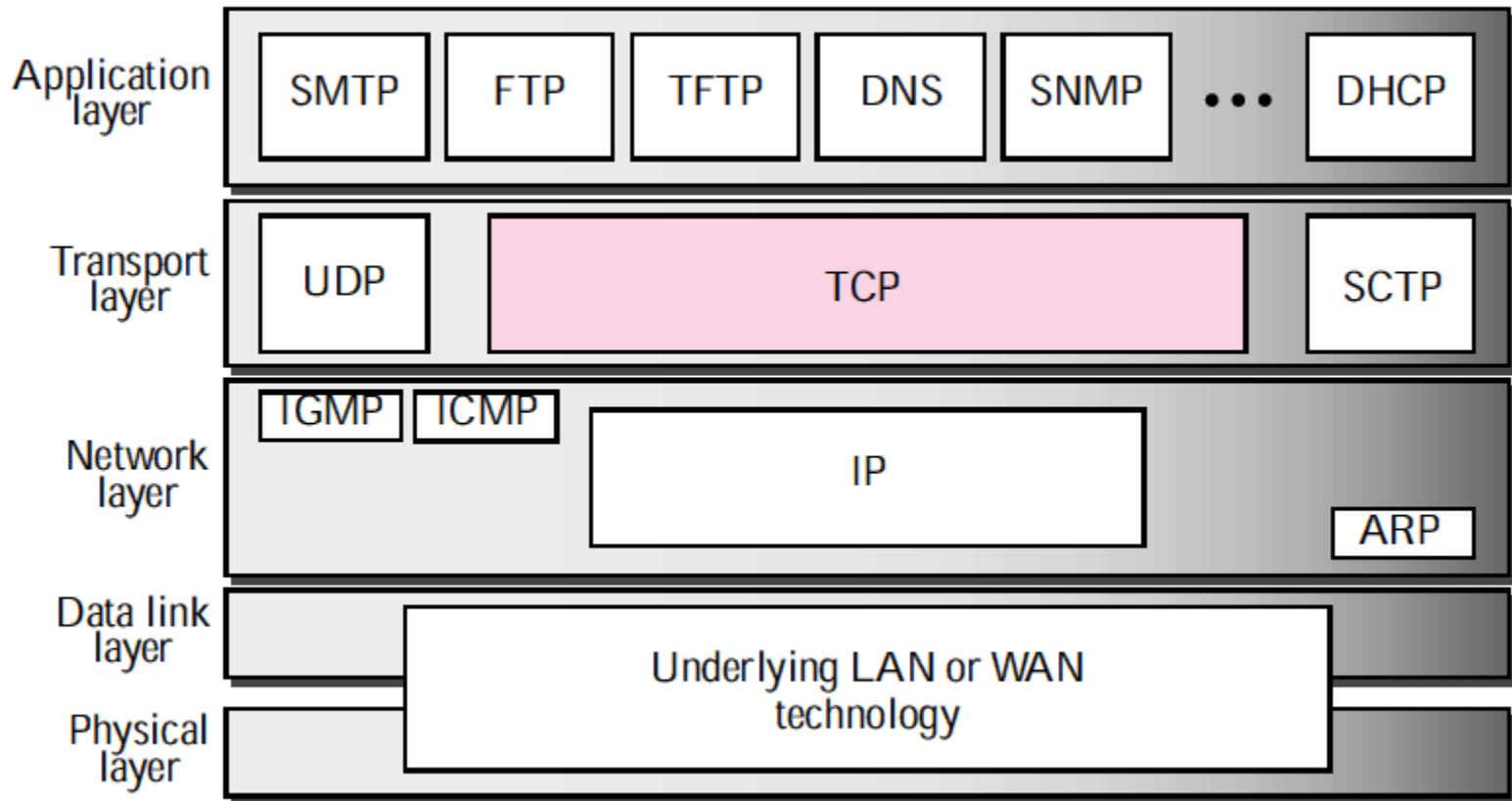
- Généralités
- UDP
- **TCP**



# TCP : Transmission Control Protocol

- Protocole orienté connexion
  - Définition explicite de l'établissement de la connexion, du transfert des données et de la terminaison de la connexion
- Transmission fiable des données :
  - Illusion assurée par le service par retransmission des paquets perdus et/ou corrompus

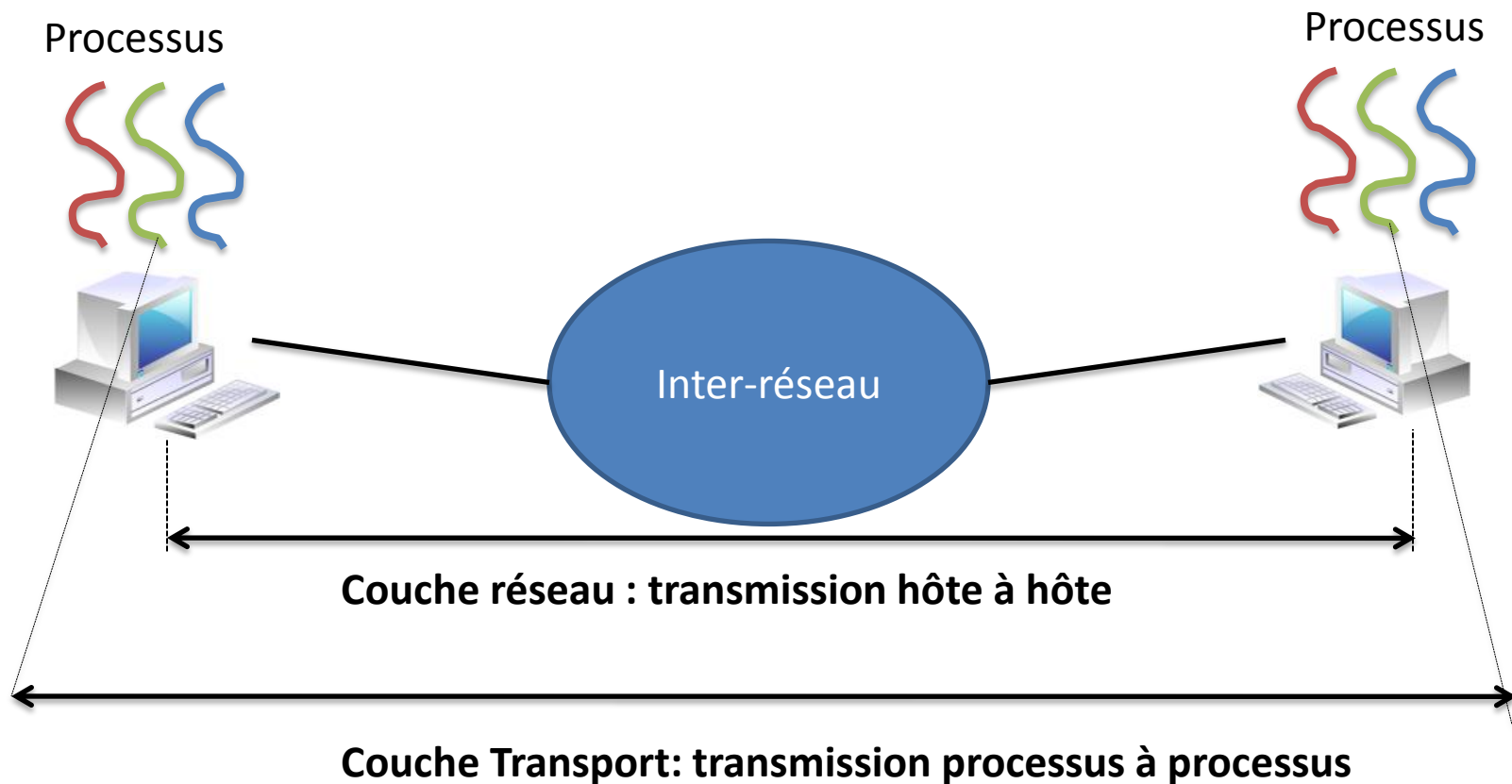
# TCP dans la pile protocolaire TCP/IP



# Services de TCP

- Communication processus-à-processus
  - Utilisation des adresses de processus : numéros de port
- Transmission en mode flux(stream) de données
  - Les données transmises sont vues comme une suite ininterrompue d'octets
- Multiplexage/Démultiplexage
- Service orienté connexion
- Communication en mode full-duplex

# Transport processus à processus

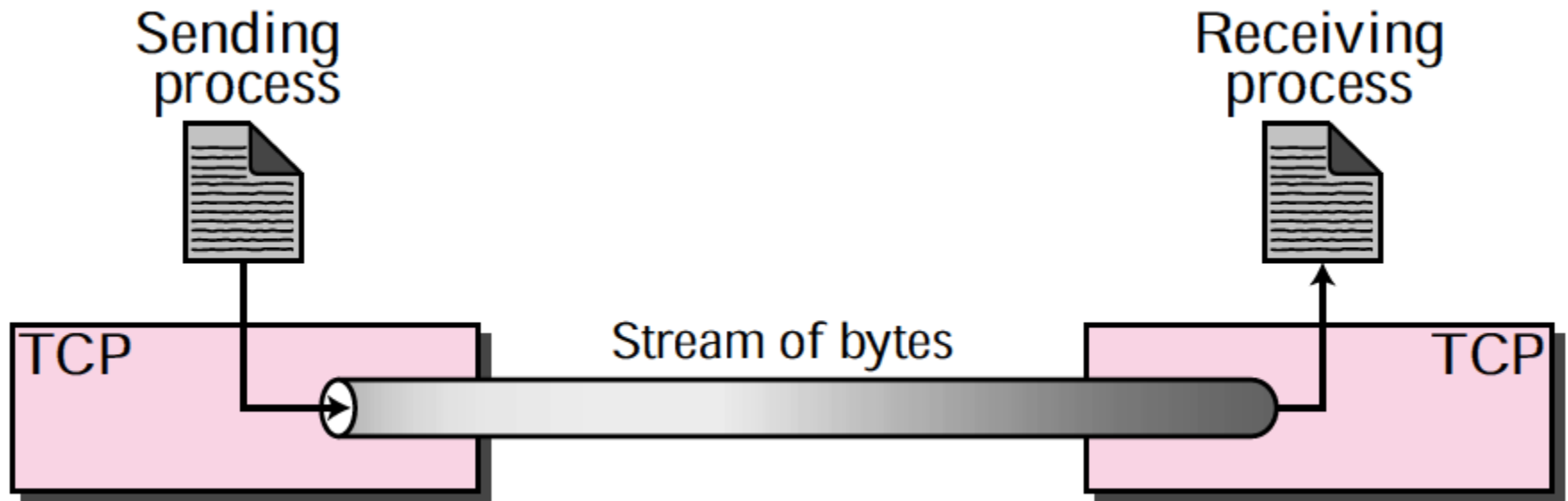


# Les applications basées sur TCP

| <i>Port</i> | <i>Protocol</i> | <i>Description</i>                            |
|-------------|-----------------|-----------------------------------------------|
| 7           | Echo            | Echoes a received datagram back to the sender |
| 9           | Discard         | Discards any datagram that is received        |
| 11          | Users           | Active users                                  |
| 13          | Daytime         | Returns the date and the time                 |
| 17          | Quote           | Returns a quote of the day                    |
| 19          | Chargen         | Returns a string of characters                |
| 20          | FTP, Data       | File Transfer Protocol (data connection)      |
| 21          | FTP, Control    | File Transfer Protocol (control connection)   |
| 23          | TELNET          | Terminal Network                              |
| 25          | SMTP            | Simple Mail Transfer Protocol                 |
| 53          | DNS             | Domain Name Server                            |
| 67          | BOOTP           | Bootstrap Protocol                            |
| 79          | Finger          | Finger                                        |
| 80          | HTTP            | Hypertext Transfer Protocol                   |
| 111         | RPC             | Remote Procedure Call                         |

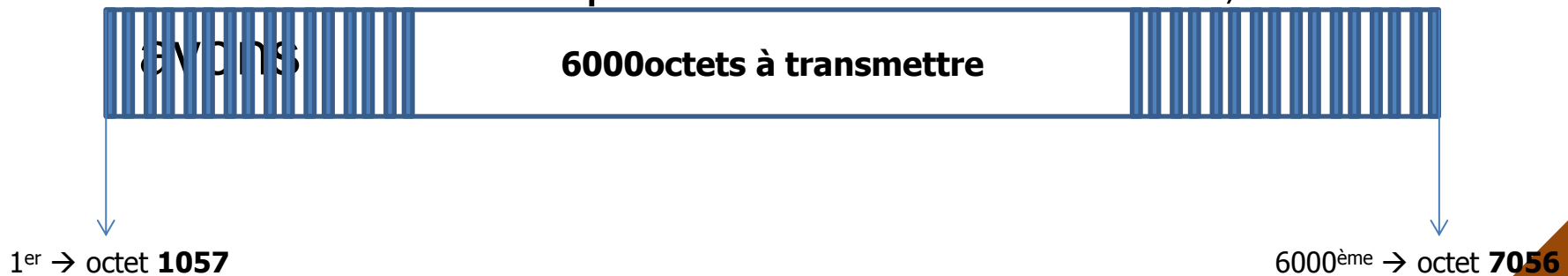
# Transmission en mode flux(stream) de données

- Les données envoyées sont considérées comme une suite d'octets ininterrompue : un **stream**



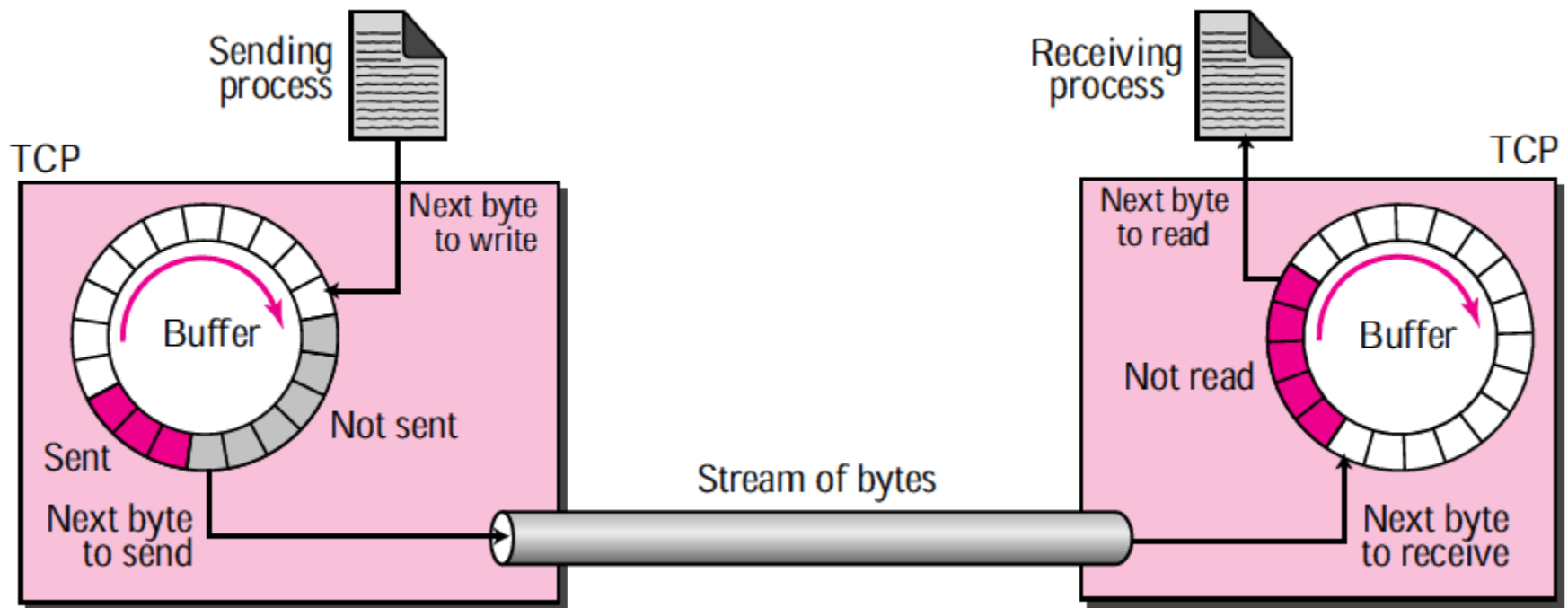
# Numérotation des octets

- TCP donne un numéro à tous les octets les uns à la suite des autres
  - Le numéro du premier octet est aléatoire entre 0 et  $2^{32}-1$  mais tous les autres sont à la suite
- Exemple :
  - si le numéro du premier octet est 1057, nous



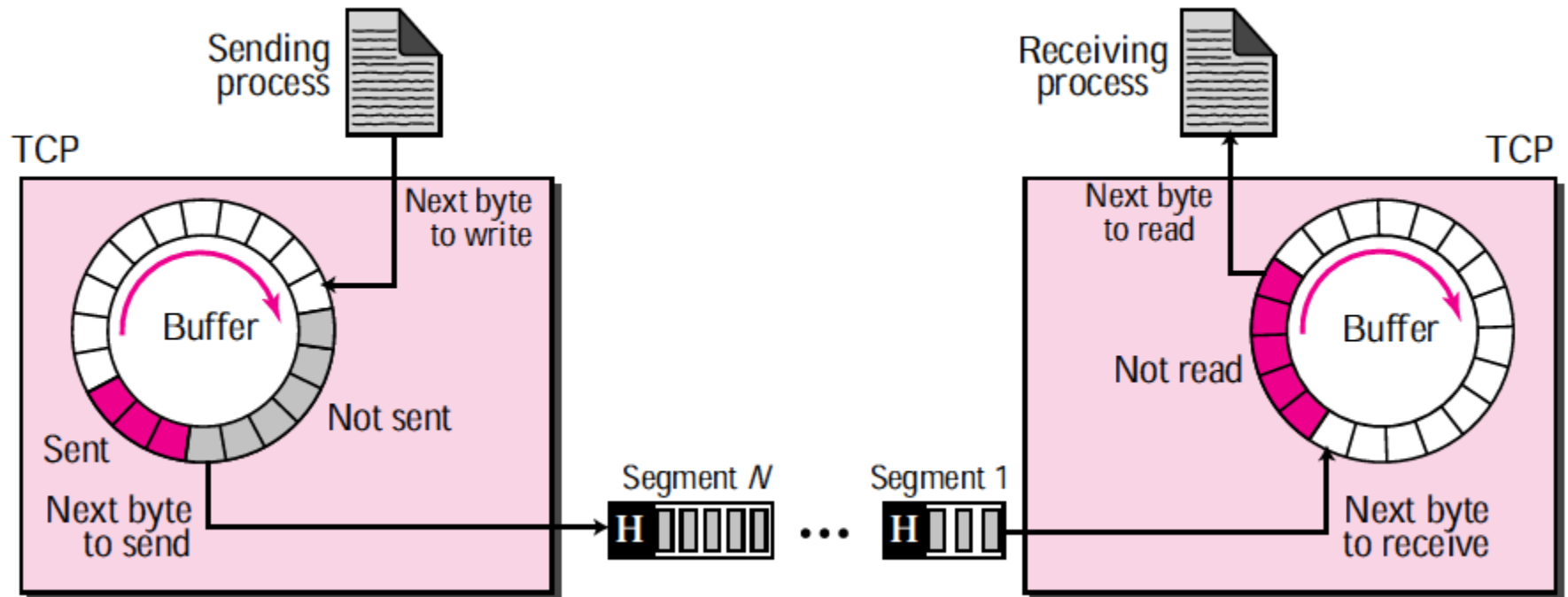
# Transmission en mode flux(stream) de données

- Les tampons d'envoi et de réception



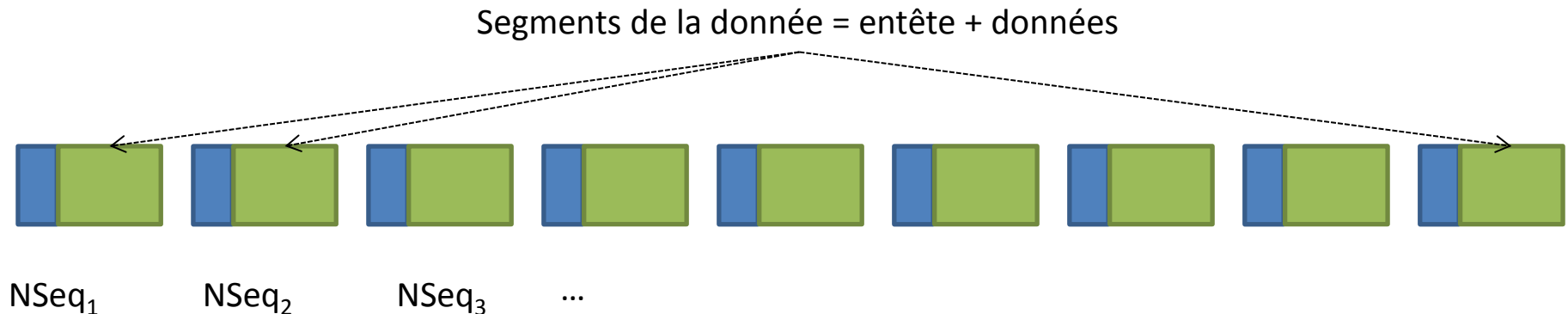
# Transmission des segments de données

- Les octets sont regroupés dans des paquets



# Numérotation des segments : numéro de séquence

- TCP donne à chaque segment un numéro
  - Représentant la position du segment dans l'ensemble des données à transmettre (du stream)
  - Ce numéro est appelé **numéro de séquence**



## Numérotation des segments : numéro de séquence

- Le numéro de séquence est défini comme suit:
  - Le numéro de séquence du premier segment (ISN: Initial Segment Number) est **un nombre aléatoire**
  - Le numéro séquence de tout autre segment est égale :
    - Au numéro de séquence du précédent segment + la quantité de données transportée par ce précédent segment

$$NSeq(Segmt_i) = NSeq(Segmt_{i-1}) + TailleData(Segmt_{i-1})$$

↓  
Numéro de  
séquence à calculer

↓  
Numéro de séquence  
du segment précédent

↓  
Taille des données transportées (en  
octet) par le segment précédent

# Exercice

Supposons une connexion TCP transmettant un fichier de 5000 octets. Le premier octet est numéroté 10001. Quel est le numéro de séquence de chaque segment si les segments transportent chacun 1000 octets de données.

## Solution

*Il y aura 5 segments*

*Segment 1 ➡ Numéro de Sequence : 10 001 (plage: 10 001 to 11 000)*

*Segment 2 ➡ Numéro de Sequence : 11 001 (plage: 11 001 to 12 000)*

*Segment 3 ➡ Numéro de Sequence : 12 001 (plage: 12 001 to 13 000)*

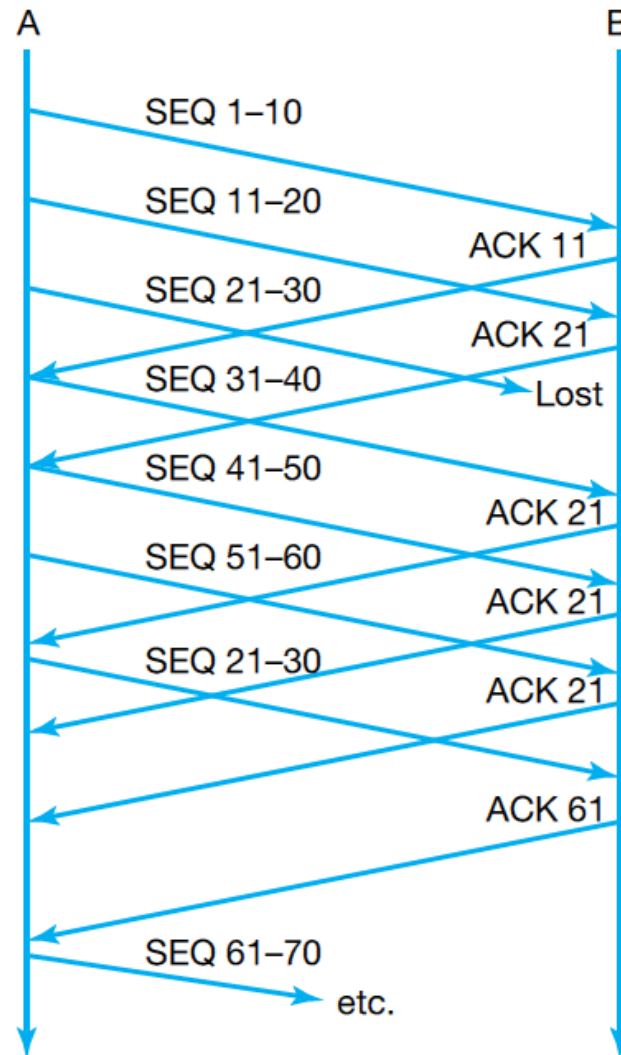
*Segment 4 ➡ Numéro de Sequence : 13 001 (plage: 13,001 to 14 000)*

*Segment 5 ➡ Numéro de Sequence : 14 001 (plage: 14 001 to 15 000)*

# Acquittement TCP

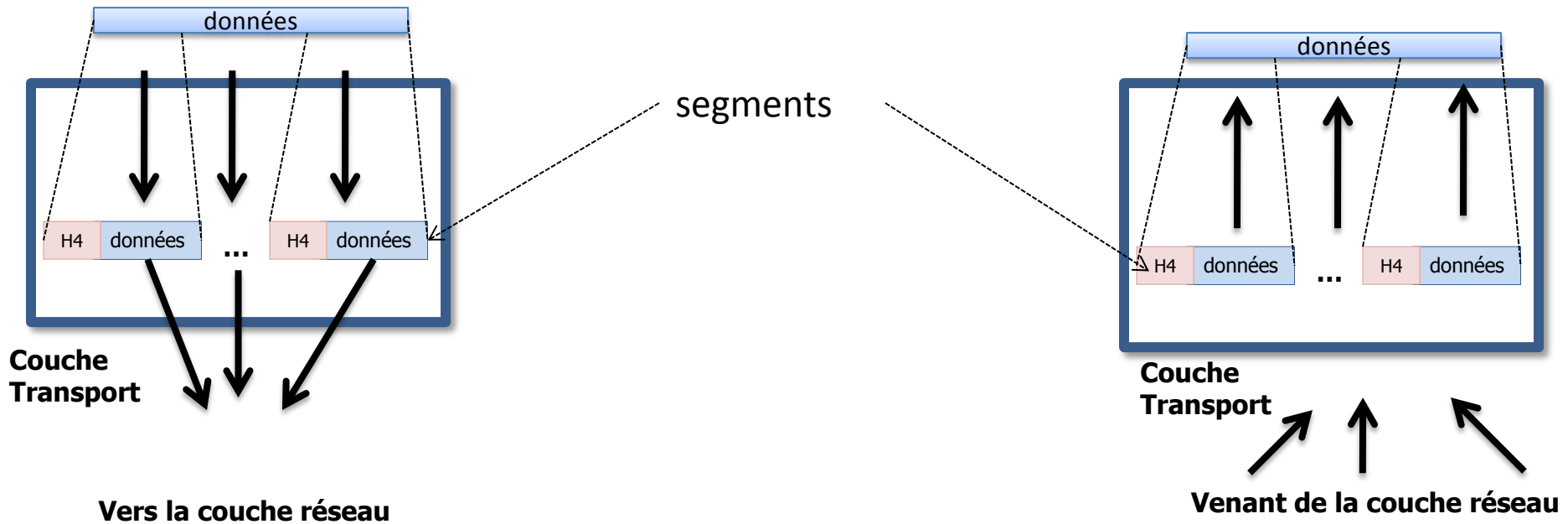
- Principe
  - Chaque segment reçu sans erreur doit être acquitté par le destinataire
  - Le numéro d'acquittement est **le numéro de séquence du prochain octet** attendu dans le stream
  - Lorsqu'un numéro de séquence est reçu, tous les octets avant ce numéro sont supposés avoir été bien reçus
- Les acquittements sont des accusés de réceptions

# Acquittement TCP



# Mise en segment de TCP

- Les données des couches hautes sont découpées en **segments** contenant les octets à transmettre

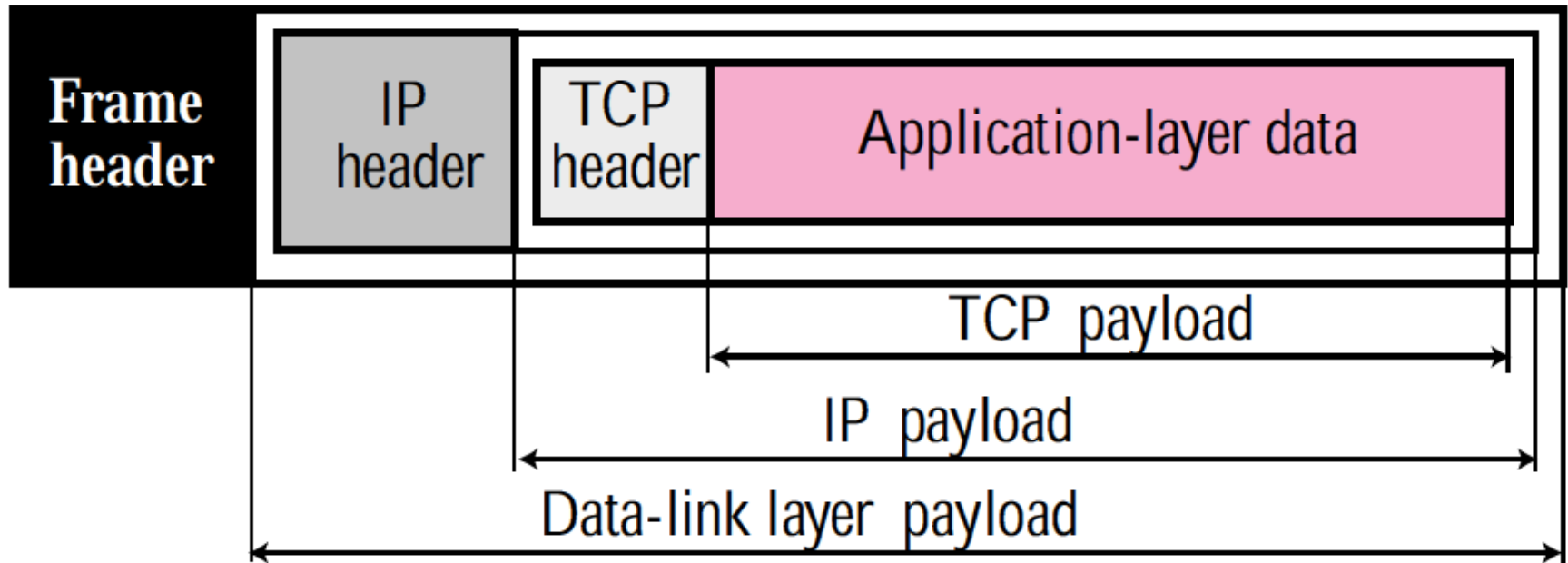


Format générique d'un segment

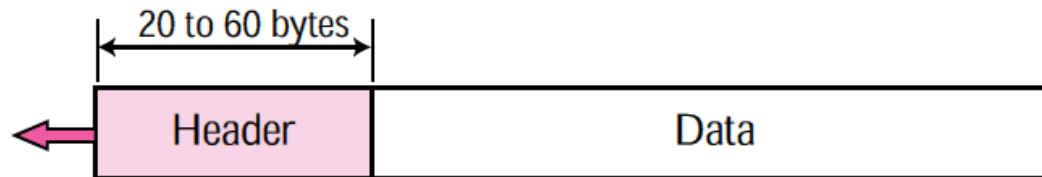
Entête TCP

Données des couches hautes

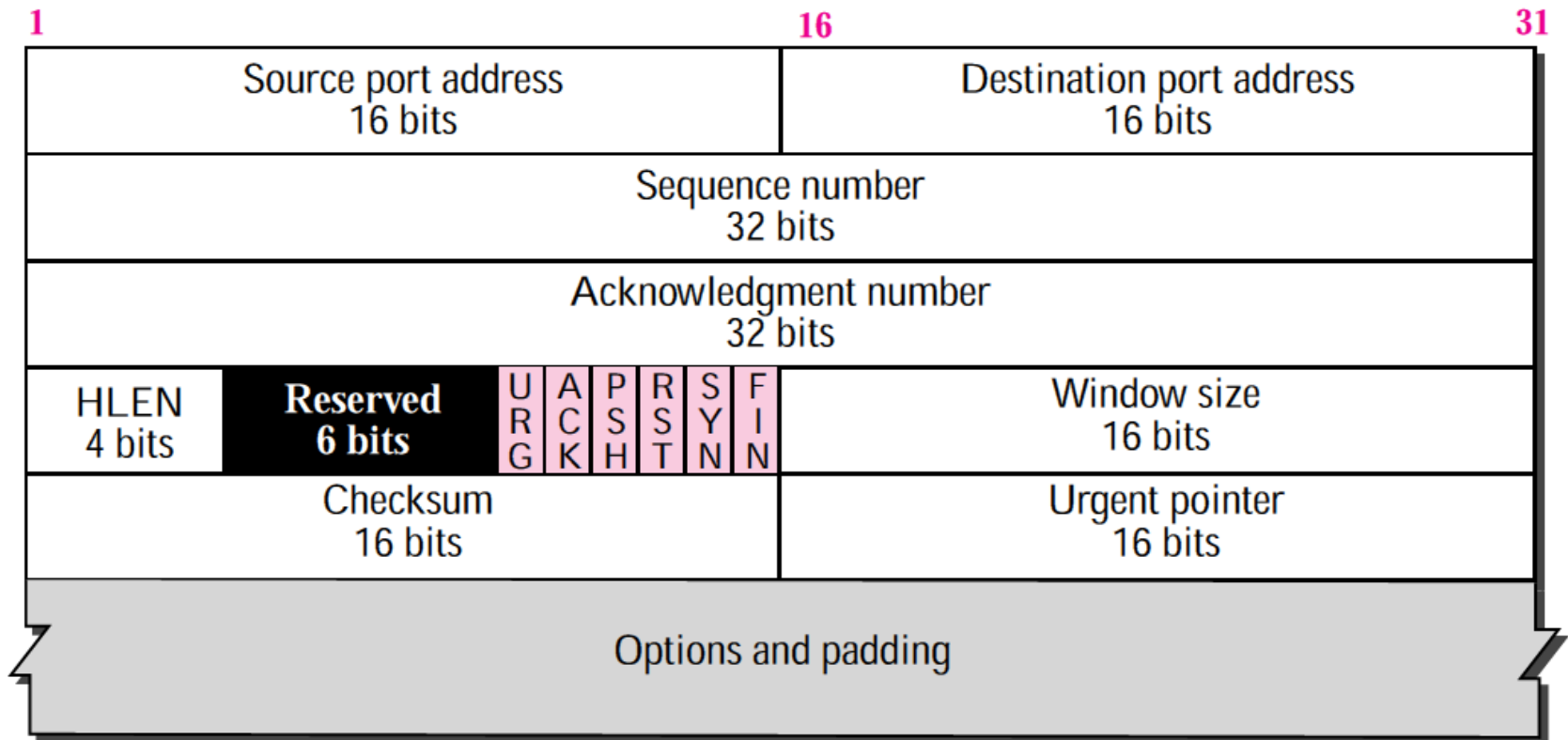
# Encapsulation TCP



# Format d'un segment TCP



a. Segment



b. Header

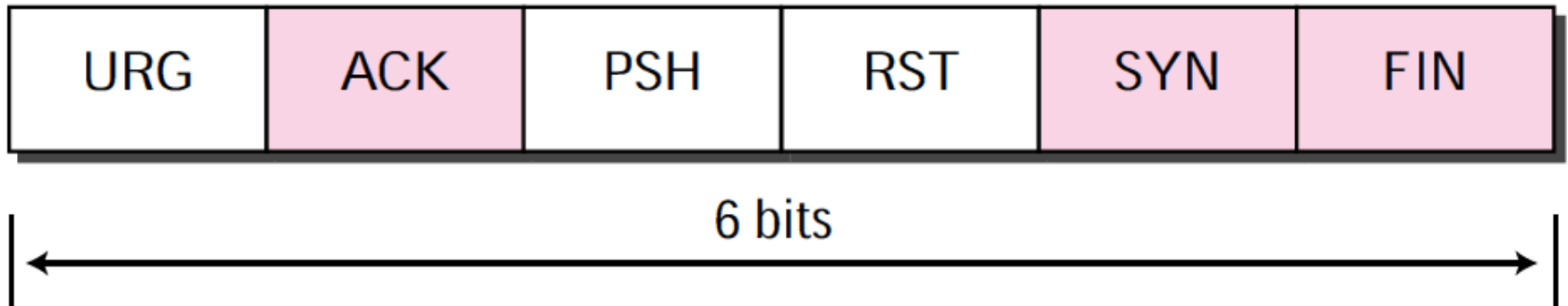
# Format d'un segment TCP

- **Numéro de port source**
  - Numéro de port du processus qui envoie le segment
  - 1024-65,535 pour le client et 1-1023 pour le serveur
- **Numéro de port destination**
  - Numéro de port du processus qui doit recevoir le segment
- **Numéro de séquence**
  - Identifie la position du segment sur le message sur 32bits
- **Numéro d'acquittement**
  - Le numéro du prochain segment que le destinataire s'attend à recevoir
- **Taille de l'entête**
  - Identifie la taille de l'entête en groupe de 4octets

# Format d'un segment TCP

- **Taille de la fenêtre**
  - Fenêtre de réception déterminée par le destinataire (rwnd : receiving window)
- **Champs de control**
  - 6 bits de contrôle pour l'établissement et la terminaison d'une connexion, le contrôle de flux
  - Chaque constitue un flag dans le processus de transfert de données

# Les flags de contrôle ou code bits



| <i>Flag</i> | <i>Description</i>                             |
|-------------|------------------------------------------------|
| URG         | The value of the urgent pointer field is valid |
| ACK         | The value of the acknowledgment field is valid |
| PSH         | Push the data                                  |
| RST         | The connection must be reset                   |
| SYN         | Synchronize sequence numbers during connection |
| FIN         | Terminate the connection                       |

# Exemple de Segment TCP

```
5 0.021673 192.168.1.100 128.121.70.151 TCP 62 netinfo-local > ftp [SYN] Seq=0 Win=64512 Len=0 MSS=1460 SACK_PERM=1
+ Frame 5: 62 bytes on wire (496 bits), 62 bytes captured (496 bits)
+ Ethernet II, Src: AmbitMic_aa:af:80 (00:d0:59:aa:af:80), Dst: Runtop_d9:0d:db (00:20:78:d9:0d:db)
+ Internet Protocol Version 4, Src: 192.168.1.100 (192.168.1.100), Dst: 128.121.70.151 (128.121.70.151)
- Transmission Control Protocol, Src Port: netinfo-local (1033), Dst Port: ftp (21), Seq: 0, Len: 0
  Source port: netinfo-local (1033)
  Destination port: ftp (21)
  [Stream index: 1]
  Sequence number: 0 (relative sequence number)
  Header length: 28 bytes
  Flags: 0x02 (SYN)
    000. .... = Reserved: Not set
    ...0 .... = Nonce: Not set
    .... 0... = Congestion window Reduced (CWR): Not set
    .... 0... = ECN-Echo: Not set
    .... ..0. = Urgent: Not set
    .... ...0 = Acknowledgement: Not set
    .... .... 0... = Push: Not set
    .... .... .0.. = Reset: Not set
    + .... .... ..1. = Syn: Set
    .... .... ...0 = Fin: Not set
  window size value: 64512
  [calculated window size: 64512]
  Checksum: 0xf8db [validation disabled]
    [Good checksum: False]
    [Bad checksum: False]
  Options: (8 bytes)
    Maximum segment size: 1460 bytes
    No-Operation (NOP)
    No-Operation (NOP)
    TCP SACK Permitted option: True

0000  00 20 78 d9 0d db 00 d0 59 aa af 80 08 00 45 00  . x..... Y.....E.
0010  00 30 00 21 40 00 80 06 71 8a c0 a8 01 64 80 79  .0.!@... q....d.y
0020  46 97 04 09 00 15 06 c3 fa 44 00 00 00 00 70 02  F..... .D....p.
0030  fc 00 f8 db 00 00 02 04 05 b4 01 01 04 02      .....

```

# Exercice

- Que pouvons nous dire des segments TCP avec les valeurs de flags suivantes

a. 0000000

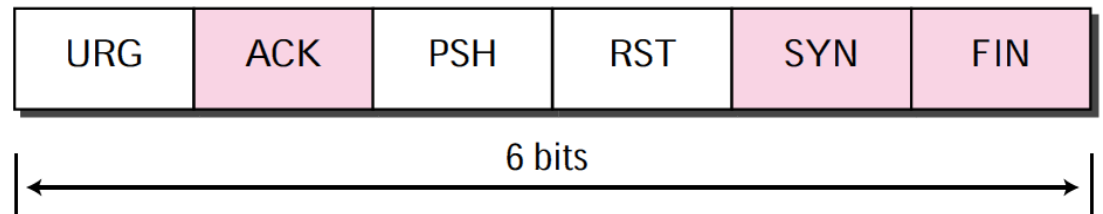
b. 0000001

c. 0100001

d. 000100

e. 000010

f. 010010

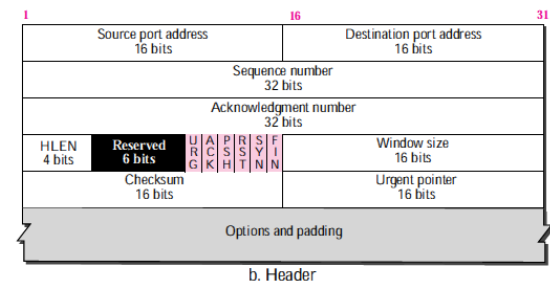


# Exercice

- Soit l'entête TCP suivant en hexadécimal

```
05320017 00000001 00000000
500207FF 00000000
```

1. Quel est le numéro de port source ?
2. Quel est le numéro de port destinataire ?
3. Quel est le numéro de séquence ?
4. Quel est le numéro d'acquittement ?
5. Quel est la taille de l'entête ?
6. Quel est le type de segment ?
7. Quel est la taille de la fenêtre



# Mécanisme de TCP

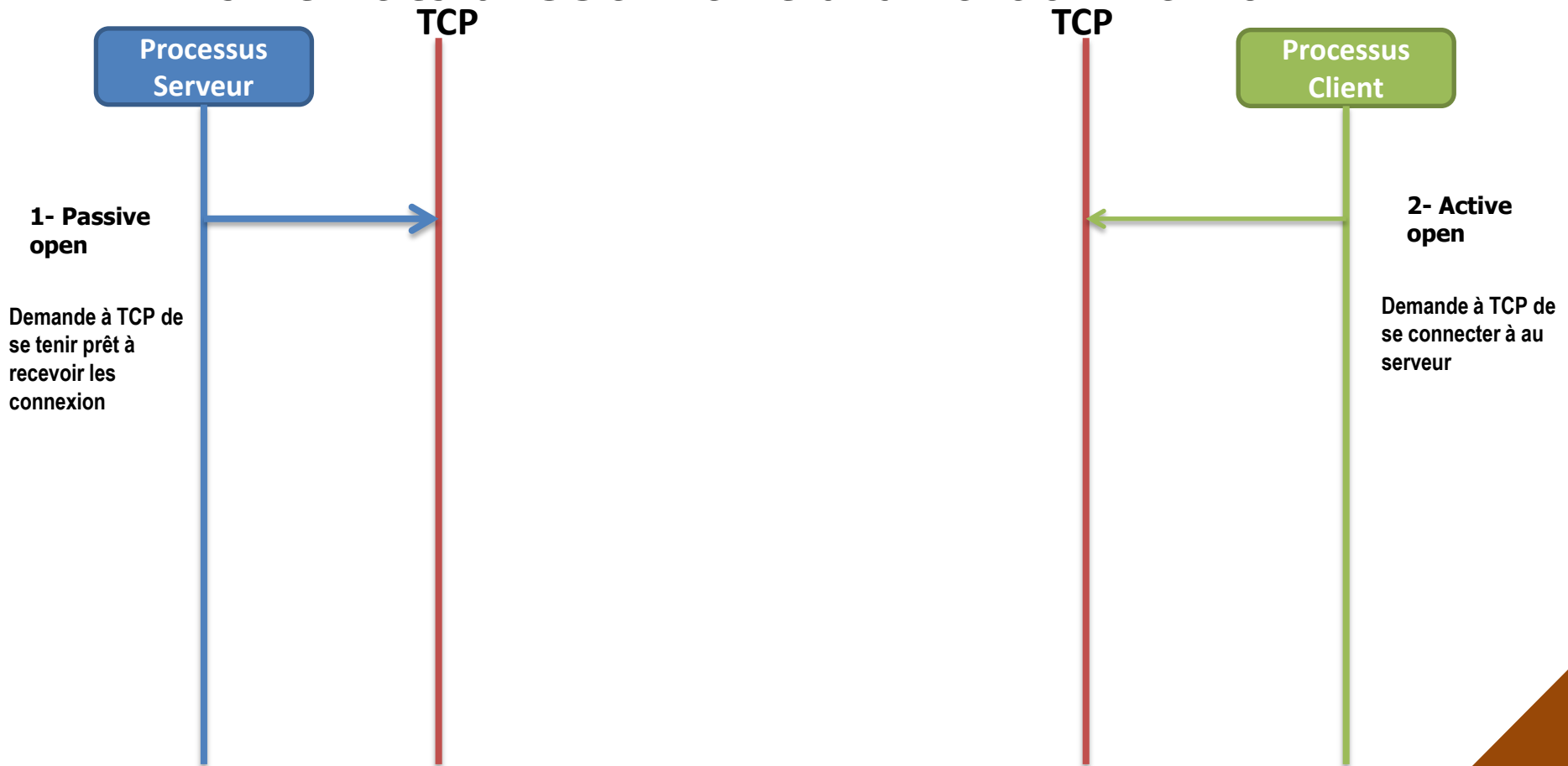
- Etablissement et terminaison d'une connexion
- Multiplexage
- Transfert de données
- Contrôle de flux
- Fiabilité
- Précédence(priorité de trafic)

# Phases de la communication TCP

- La connexion TCP est un chemin virtuel entre la source et la destination
- TCP définit les différentes phases d'une communication
  - Connexion
  - Transfert de données en full-duplex(simultanément dans les deux sens)
  - Déconnexion

# Etablissement de connexion TCP

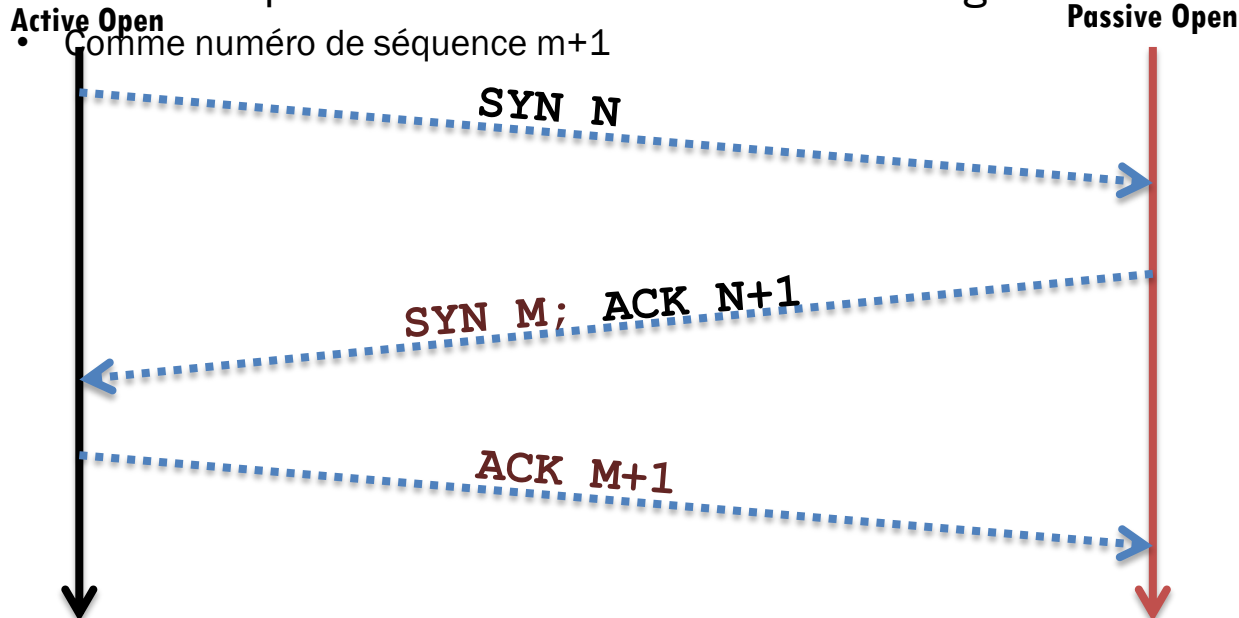
- Avant l'établissement d'une connexion



# Etablissement de connexion TCP

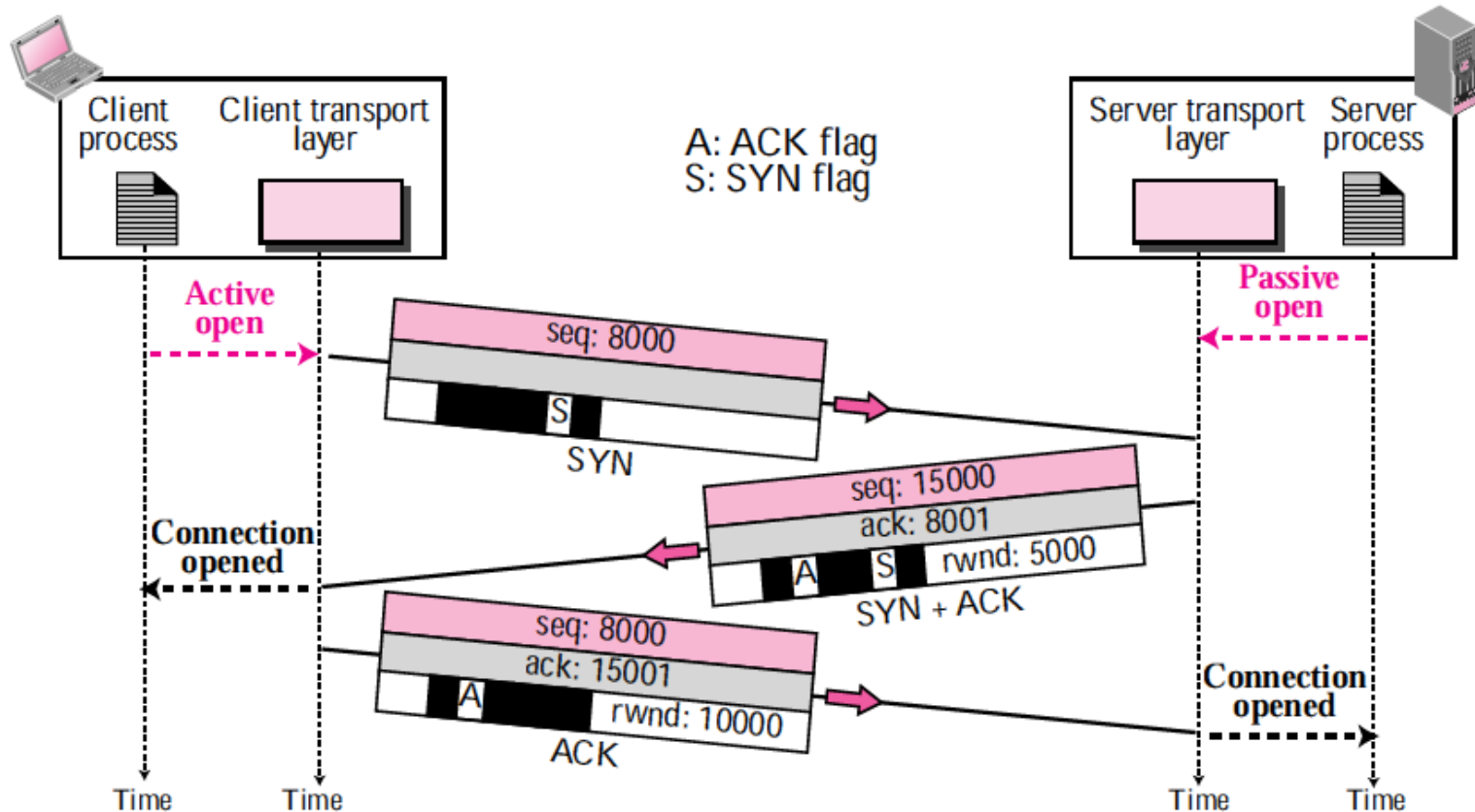
- Principe

- L'initiateur envoie un segment SYN (segment dont le flag SYN est activé)
  - Il choisit aléatoirement son ISN (Initial Sequence Number)  $N$
- Le récepteur répond par un segment SYN :
  - Contenant son propre ISN  $M$
  - Il acquitte le segment SYN de l'initiateur avec comme numéro de séquence  $N+1$
- L'initiateur acquitte le SYN du serveur avec un segment ACK



# Etablissement de connexion TCP

- Le processus d'établissement de la connexion proprement dit TCP est connu sous l'appellation **Three-Way-Handshake**
  - Poignée de main en trois temps



## Three Way handshake et Numéro de séquence

- Règles des numéros de séquences :
  - Un paquet SYN ne peut transporter de données mais consomme un numéro de séquence (1octet)
  - Un paquet SYN+ACK ne transporte pas de données mais consomme un numéro de séquence
  - Un paquet ACK ne consomme 1 numéro de séquence que lorsqu'elle transport une donnée

# Exemple de 3-way-Handshake

ftp-port-ok.pcap [Wireshark 1.6.0 (SVN Rev 37592 from /trunk-1.6)]

File Edit View Go Capture Analyze Statistics Telephony Tools Internals Help

Filter: tcp.flags.syn==1 || tcp.flags.ack==1 Expression... Clear Apply

| No. | Time      | Source         | Destination    | Protocol | Length | Info                                                                                 |
|-----|-----------|----------------|----------------|----------|--------|--------------------------------------------------------------------------------------|
| 5   | 0.021673  | 192.168.1.100  | 128.121.70.151 | TCP      | 62     | 1033 > 21 [SYN] Seq=0 win=64512 Len=0 MSS=1460 SACK_PERM=1                           |
| 6   | 0.123207  | 128.121.70.151 | 192.168.1.100  | TCP      | 62     | 21 > 1033 [SYN, ACK] Seq=0 Ack=1 win=49152 Len=0 MSS=1460 SACK_PERM=1                |
| 7   | 0.123295  | 192.168.1.100  | 128.121.70.151 | TCP      | 60     | 1033 > 21 [ACK] Seq=1 Ack=1 win=64512 Len=0                                          |
| 9   | 0.472907  | 192.168.1.100  | 128.121.70.151 | TCP      | 60     | 1033 > 21 [ACK] Seq=1 Ack=18 win=64495 Len=0                                         |
| 10  | 3.421830  | 192.168.1.100  | 128.121.70.151 | FTP      | 70     | Request: USER anonymous                                                              |
| 11  | 3.521697  | 128.121.70.151 | 192.168.1.100  | TCP      | 60     | 21 > 1033 [ACK] Seq=18 Ack=17 win=49152 Len=0                                        |
| 12  | 3.548423  | 128.121.70.151 | 192.168.1.100  | FTP      | 130    | Response: 331 Anonymous login ok, send your complete email address as your password. |
| 13  | 3.677512  | 192.168.1.100  | 128.121.70.151 | TCP      | 60     | 1033 > 21 [ACK] Seq=17 Ack=94 win=64419 Len=0                                        |
| 14  | 10.375572 | 192.168.1.100  | 128.121.70.151 | FTP      | 73     | Request: PASS telegraph23                                                            |
| 15  | 10.477554 | 128.121.70.151 | 192.168.1.100  | TCP      | 60     | 21 > 1033 [ACK] Seq=94 Ack=36 win=49152 Len=0                                        |
| 16  | 10.507648 | 128.121.70.151 | 192.168.1.100  | FTP      | 105    | Response: 230 Anonymous access granted, restrictions apply.                          |
| 17  | 10.687595 | 192.168.1.100  | 128.121.70.151 | TCP      | 60     | 1033 > 21 [ACK] Seq=36 Ack=145 win=64368 Len=0                                       |
| 18  | 12.514844 | 192.168.1.100  | 128.121.70.151 | FTP      | 79     | Request: PORT 192,168,1,100,4,10                                                     |
| 19  | 12.623594 | 128.121.70.151 | 192.168.1.100  | TCP      | 60     | 21 > 1033 [ACK] Seq=145 Ack=61 win=49152 Len=0                                       |
| 20  | 12.624093 | 128.121.70.151 | 192.168.1.100  | FTP      | 84     | Response: 200 PORT command successful.                                               |
| 21  | 12.624768 | 192.168.1.100  | 128.121.70.151 | FTP      | 60     | Request: NLST                                                                        |
| 22  | 12.725483 | 128.121.70.151 | 192.168.1.100  | TCP      | 60     | 21 > 1033 [ACK] Seq=175 Ack=67 win=49152 Len=0                                       |
| 23  | 12.727982 | 128.121.70.151 | 192.168.1.100  | TCP      | 62     | 20 > 1034 [SYN] Seq=0 win=49152 Len=0 MSS=1460 SACK_PERM=1                           |
| 24  | 12.728027 | 192.168.1.100  | 128.121.70.151 | TCP      | 62     | 1034 > 20 [SYN, ACK] Seq=0 Ack=1 win=64512 Len=0 MSS=1460 SACK_PERM=1                |
| 25  | 12.828740 | 128.121.70.151 | 192.168.1.100  | TCP      | 60     | 20 > 1034 [ACK] Seq=1 Ack=1 win=49152 Len=0                                          |
| 26  | 12.829264 | 128.121.70.151 | 192.168.1.100  | FTP      | 109    | Response: 150 opening ASCII mode data connection for file list.                      |
| 27  | 12.829598 | 128.121.70.151 | 192.168.1.100  | FTP-DAT/ | 60     | FTP Data: 3 bytes                                                                    |
| 28  | 12.831546 | 128.121.70.151 | 192.168.1.100  | FTP-DAT/ | 103    | FTP Data: 49 bytes                                                                   |
| 29  | 12.831605 | 192.168.1.100  | 128.121.70.151 | TCP      | 60     | 1034 > 20 [ACK] Seq=1 Ack=54 win=64460 Len=0                                         |
| 30  | 12.832086 | 192.168.1.100  | 128.121.70.151 | TCP      | 60     | 1034 > 20 [FIN, ACK] Seq=1 Ack=54 win=64460 Len=0                                    |
| 31  | 12.934789 | 128.121.70.151 | 192.168.1.100  | TCP      | 60     | 20 > 1034 [ACK] Seq=54 Ack=2 win=49152 Len=0                                         |
| 32  | 12.990883 | 128.121.70.151 | 128.121.70.151 | TCP      | 60     | 1033 > 21 [ACK] Seq=67 Ack=230 win=64283 Len=0                                       |
| 33  | 13.091542 | 128.121.70.151 | 192.168.1.100  | FTP      | 78     | Response: 226 Transfer complete.                                                     |
| 34  | 13.291330 | 192.168.1.100  | 128.121.70.151 | TCP      | 60     | 1033 > 21 [ACK] Seq=67 Ack=254 win=64259 Len=0                                       |
| 35  | 18.512056 | 192.168.1.100  | 128.121.70.151 | FTP      | 79     | Request: PORT 192,168,1,100,4,11                                                     |

0000 00 20 78 d9 0d db 00 d0 59 aa af 80 08 00 45 00 . x..... Y.....E.  
0010 00 30 00 21 40 00 80 06 71 8a c0 a8 01 64 80 79 .0.!@... q....d.y  
0020 46 97 04 09 00 15 06 c3 fa 44 00 00 00 70 02 F..... .D....p.  
0030 fc 00 f8 db 00 00 02 04 05 b4 01 01 04 02 .....

# Exemple de 3-way-Handshake

5 0.021673 192.168.1.100 128.121.70.151 TCP 62 1033 > 21 [SYN] Seq=0 Win=64512 Len=0 MSS=1460 SACK\_PERM=1

Frame 5: 62 bytes on wire (496 bits), 62 bytes captured (496 bits)

Ethernet II, Src: 00:d0:59:aa:af:80 (00:d0:59:aa:af:80), Dst: 00:20:78:d9:0d:db (00:20:78:d9:0d:db)

Internet Protocol Version 4, Src: 192.168.1.100 (192.168.1.100), Dst: 128.121.70.151 (128.121.70.151)

Transmission Control Protocol, Src Port: 1033 (1033), Dst Port: 21 (21), Seq: 0, Len: 0

Source port: 1033 (1033)

Destination port: 21 (21)

[Stream index: 1]

Sequence number: 0 (relative sequence number)

Header length: 28 bytes

Flags: 0x02 (SYN)

000. .... = Reserved: Not set

...0 .... = Nonce: Not set

.... 0... = Congestion Window Reduced (CWR): Not set

.... .0.. = ECN-Echo: Not set

.... ..0. = Urgent: Not set

.... ...0 = Acknowledgement: Not set

.... .... 0... = Push: Not set

.... ..... 0.. = Reset: Not set

.... .... .1. = Syn: Set

[Expert Info (Chat/Sequence): Connection establish request (SYN): server port 21]

[Message: Connection establish request (SYN): server port 21]

[Severity level: chat]

[Group: Sequence]

.... .... .0 = Fin: Not set

Window size value: 64512

[Calculated window size: 64512]

Checksum: 0xf8db [validation disabled]

[Good checksum: False]

[Bad checksum: False]

Options: (8 bytes)

Maximum segment size: 1460 bytes

No-Operation (NOP)

No-Operation (NOP)

TCP SACK Permitted Option: True

Le flag SYN est activé

0000 00 20 78 d9 0d db 00 d0 59 aa af 80 08 00 45 00 . x.....Y.....E.

0010 00 30 00 21 40 00 80 06 71 8a c0 a8 01 64 80 79 .0.!@...q....d.y

0020 46 97 04 09 00 15 06 c3 fa 44 00 00 00 70 02 F.....D....p.

0030 fc 00 f8 db 00 00 02 04 05 b4 01 01 04 02 ..... .

# Exemple de 3-way Handshake

6 0.123207 128.121.70.151 192.168.1.100 TCP 62 21 > 1033 [SYN, ACK] Seq=0 Ack=1 Win=49152 Len=0 MSS=1460 SACK\_PERM...

Ethernet II, Src: 00:20:78:09:00:00 (00:20:78:09:00:00), Dst: 00:00:39:aa:af:20 (00:00:39:aa:af:20)

Internet Protocol Version 4, Src: 128.121.70.151 (128.121.70.151), Dst: 192.168.1.100 (192.168.1.100)

Transmission Control Protocol, Src Port: 21 (21), Dst Port: 1033 (1033), Seq: 0, Ack: 1, Len: 0

Source port: 21 (21)  
Destination port: 1033 (1033)  
[Stream index: 1]  
Sequence number: 0 (relative sequence number)  
Acknowledgement number: 1 (relative ack number)  
Header length: 28 bytes

Flags: 0x12 (SYN, ACK)

000. .... = Reserved: Not set  
...0 .... = Nonce: Not set  
.... 0... = Congestion Window Reduced (CWR): Not set  
.... .0.. = ECN-Echo: Not set  
.... ..0. = Urgent: Not set  
.... ...1 .... = Acknowledgement: Set  
.... .... 0... = Push: Not set  
.... .... .0.. = Reset: Not set  
.... .... ..1. = Syn: Set

[Expert Info (Chat/Sequence): Connection establish acknowledge (SYN+ACK): server port 21]  
[Message: Connection establish acknowledge (SYN+ACK): server port 21]  
[Severity level: Chat]  
[Group: Sequence]  
.... .... ...0 = Fin: Not set  
window size value: 49152  
[calculated window size: 49152]  
Checksum: 0x29eb [validation disabled]  
[Good checksum: False]  
[Bad checksum: False]  
Options: (8 bytes)  
Maximum segment size: 1460 bytes  
No-operation (NOP)  
No-operation (NOP)  
TCP SACK Permitted Option: True  
[SEQ/ACK analysis]  
[\[This is an ACK to the segment in frame: 5\]](#)  
[The RTT to ACK the segment was: 0.101534000 seconds]

0000 00 d0 59 aa af 80 00 20 78 d9 0d db 08 00 45 00 ..Y.... X....E.  
0010 00 30 fc c0 40 00 29 06 cb ea 80 79 46 97 c0 a8 .0..@.)...yF..  
0020 01 64 00 15 04 09 40 03 ca dc 06 c3 fa 45 70 12 .d....@. ....Ep  
0030 c0 00 29 eb 00 00 02 04 05 b4 01 01 04 02 ..).....

Le flag SYN est  
activé

Le flag ACK est activé

# Exemple de 3-way-Handshake

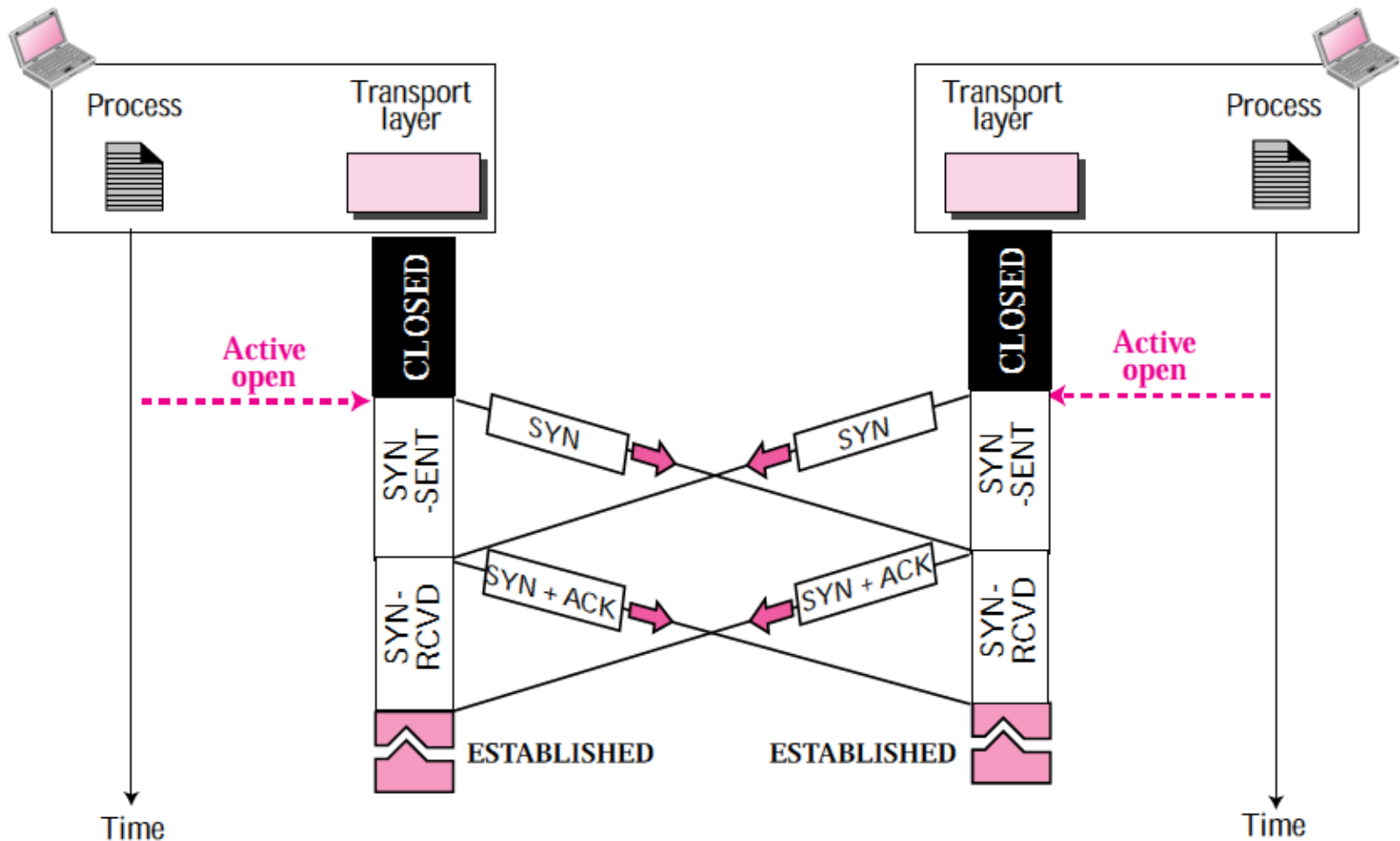


```
7 0.123295 192.168.1.100 128.121.70.151 TCP 60 1033 > 21 [ACK] Seq=1 Ack=1 Win=64512 Len=0
+ Frame 7: 60 bytes on wire (480 bits), 60 bytes captured (480 bits)
+ Ethernet II, Src: 00:d0:59:aa:af:80 (00:d0:59:aa:af:80), Dst: 00:20:78:d9:0d:db (00:20:78:d9:0d:db)
+ Internet Protocol Version 4, Src: 192.168.1.100 (192.168.1.100), Dst: 128.121.70.151 (128.121.70.151)
+ Transmission Control Protocol, Src Port: 1033 (1033), Dst Port: 21 (21), Seq: 1, Ack: 1, Win: 64512, Len: 0
  Source port: 1033 (1033)
  Destination port: 21 (21)
  [Stream index: 1]
  Sequence number: 1 (relative sequence number)
  Acknowledgement number: 1 (relative ack number)
  Header length: 20 bytes
  Flags: 0x10 (ACK)
    000. .... = Reserved: Not set
    ...0 .... = Nonce: Not set
    .... 0... = Congestion window Reduced (CWR): Not set
    .... .0.. = ECN-Echo: Not set
    .... ..0. = Urgent: Not set
    .... ...1 .... = Acknowledgement: Set
    .... .... 0... = Push: Not set
    .... .... .0.. = Reset: Not set
    .... .... ..0. = Syn: Not set
    .... .... ...0 = Fin: Not set
  window size value: 64512
  [calculated window size: 64512]
  [window size scaling factor: -2 (no window scaling used)]
  Checksum: 0x1aaf [validation disabled]
    [Good Checksum: False]
    [Bad Checksum: False]
  [SEQ/ACK analysis]
    [This is an ACK to the segment in frame: 6]
    [The RTT to ACK the segment was: 0.000088000 seconds]
```

|      |                                                 |                     |
|------|-------------------------------------------------|---------------------|
| 0000 | 00 20 78 d9 0d db 00 d0 59 aa af 80 08 00 45 00 | . x..... Y.....E.   |
| 0010 | 00 28 00 22 40 00 80 06 71 91 c0 a8 01 64 80 79 | . (. "@... q....d.y |
| 0020 | 46 97 04 09 00 15 06 c3 fa 45 40 03 ca dd 50 10 | F..... .E@...P      |
| 0030 | fc 00 1a af 00 00 00 00 00 00 00 00 00 00 00    | .....               |

# Ouverture TCP par segments SYN croisés

- La connexion est ouverte par envoi simultané de SYN et SYN+ACK dans

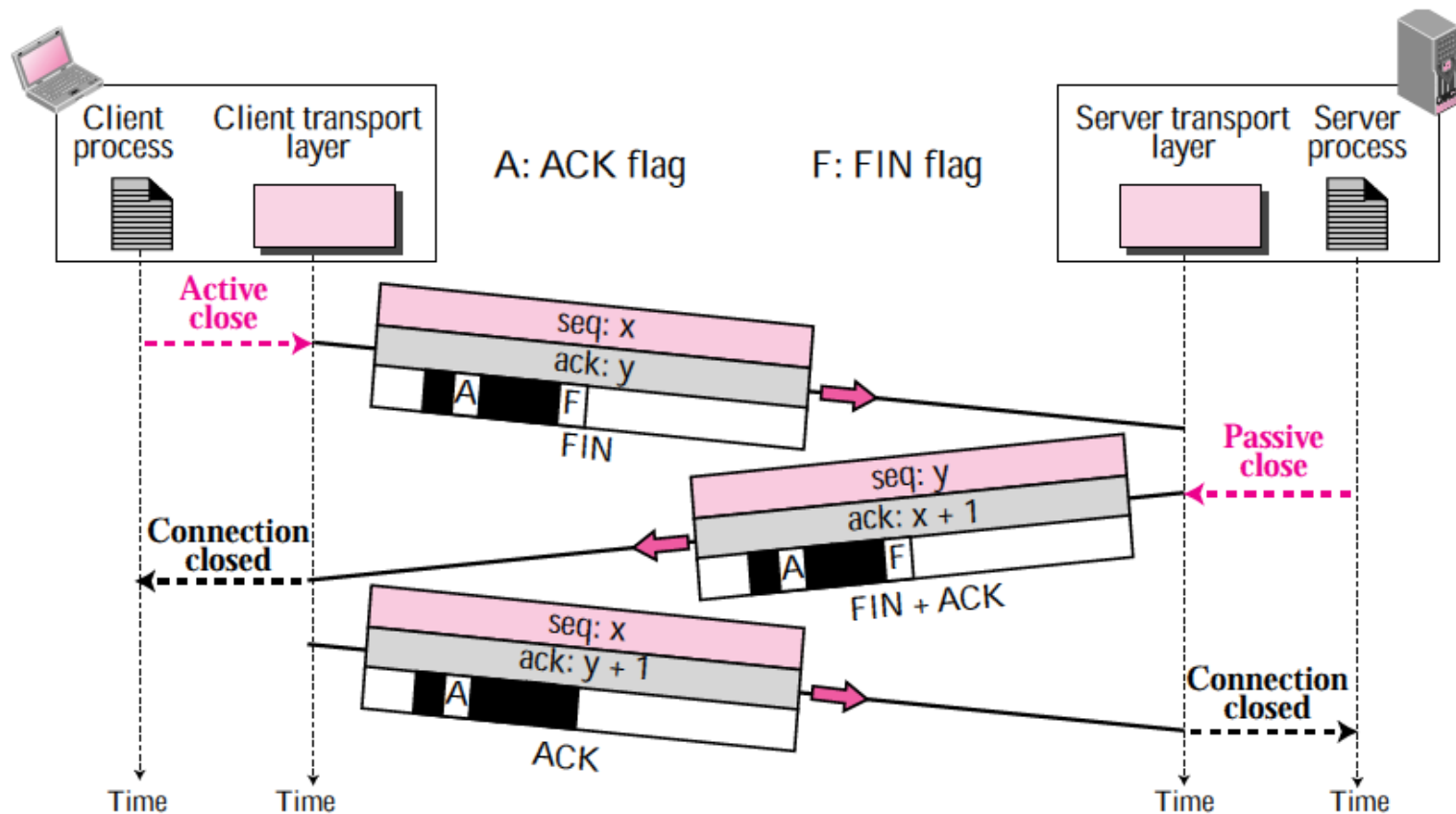


# Terminaison de la connexion

- La terminaison de la connexion peut aussi être un processus à trois temps
  - La source envoie un segment **FIN**
  - La destination répond par un segment **FIN/ACK**
  - La source renvoie un **ACK** pour confirmation de la terminaison de la session

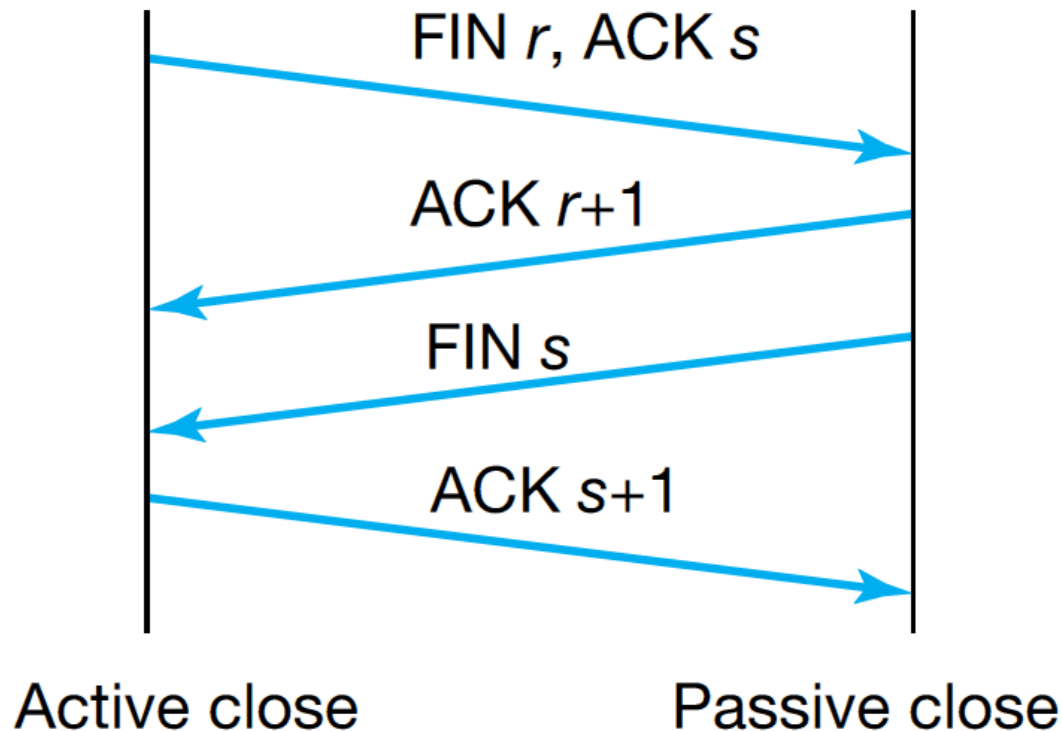
# Terminaison de la connexion

- Terminaison à trois temps



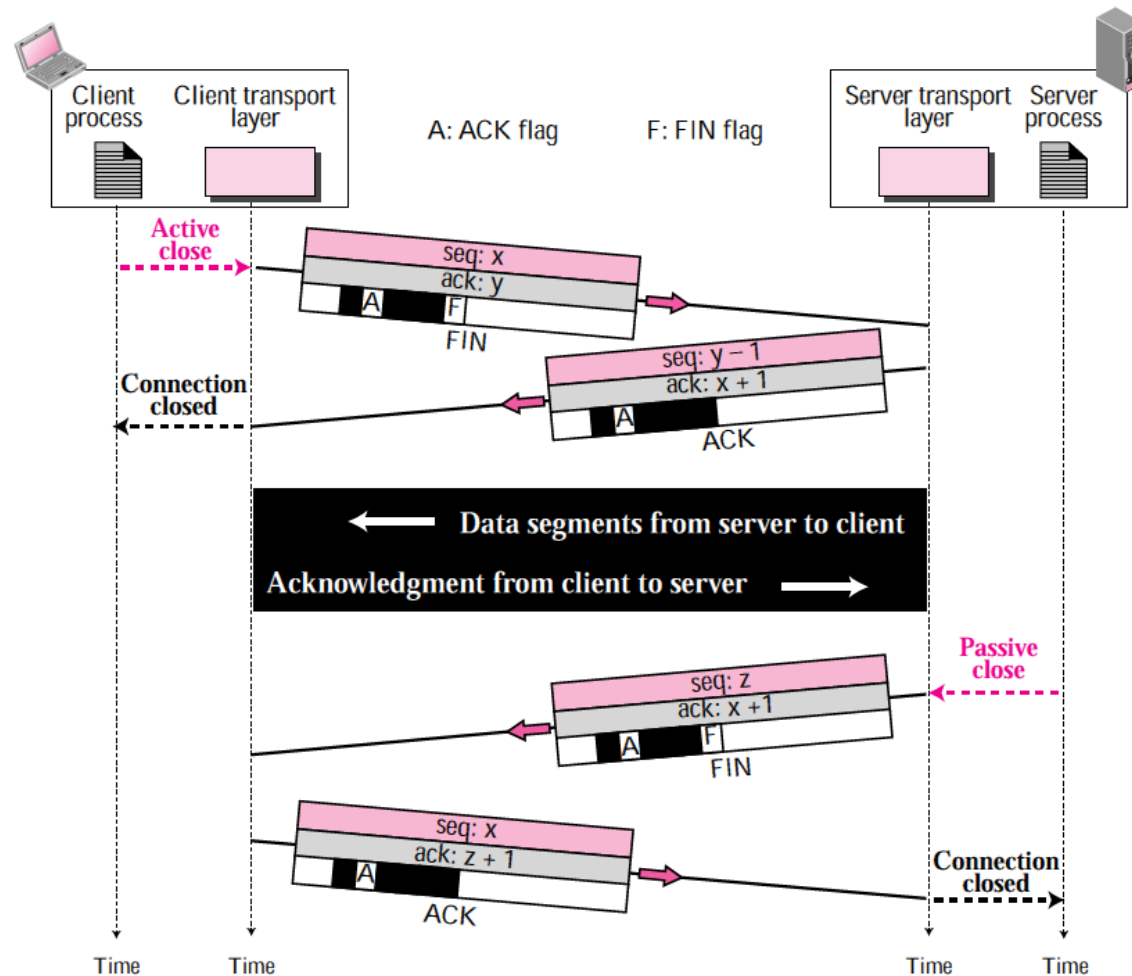
# Terminaison de la connexion

- Terminaison normale : à quatre temps
  - C'e



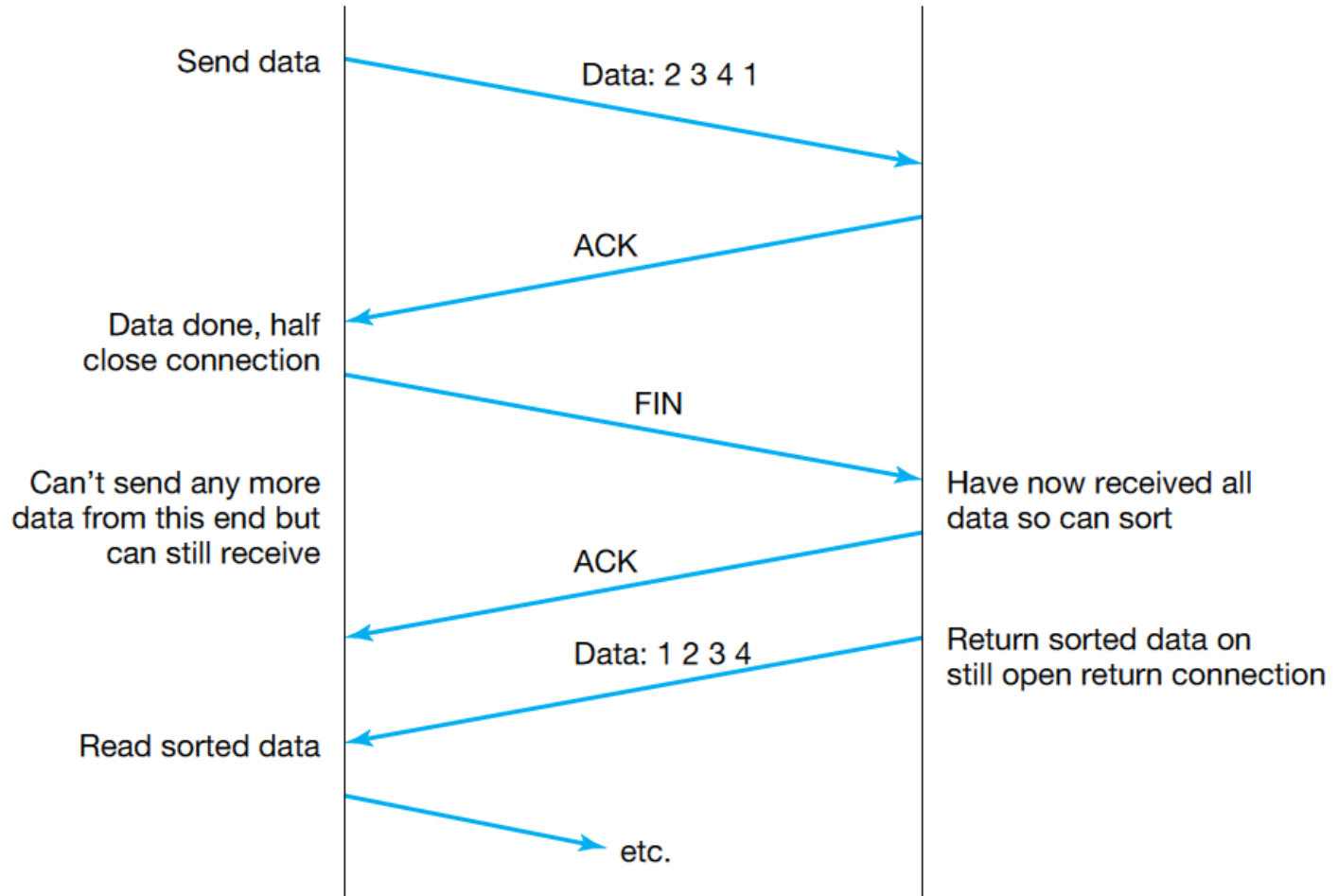
# Terminaison de la connexion

- Fermeture à moitié : TCP Half-close

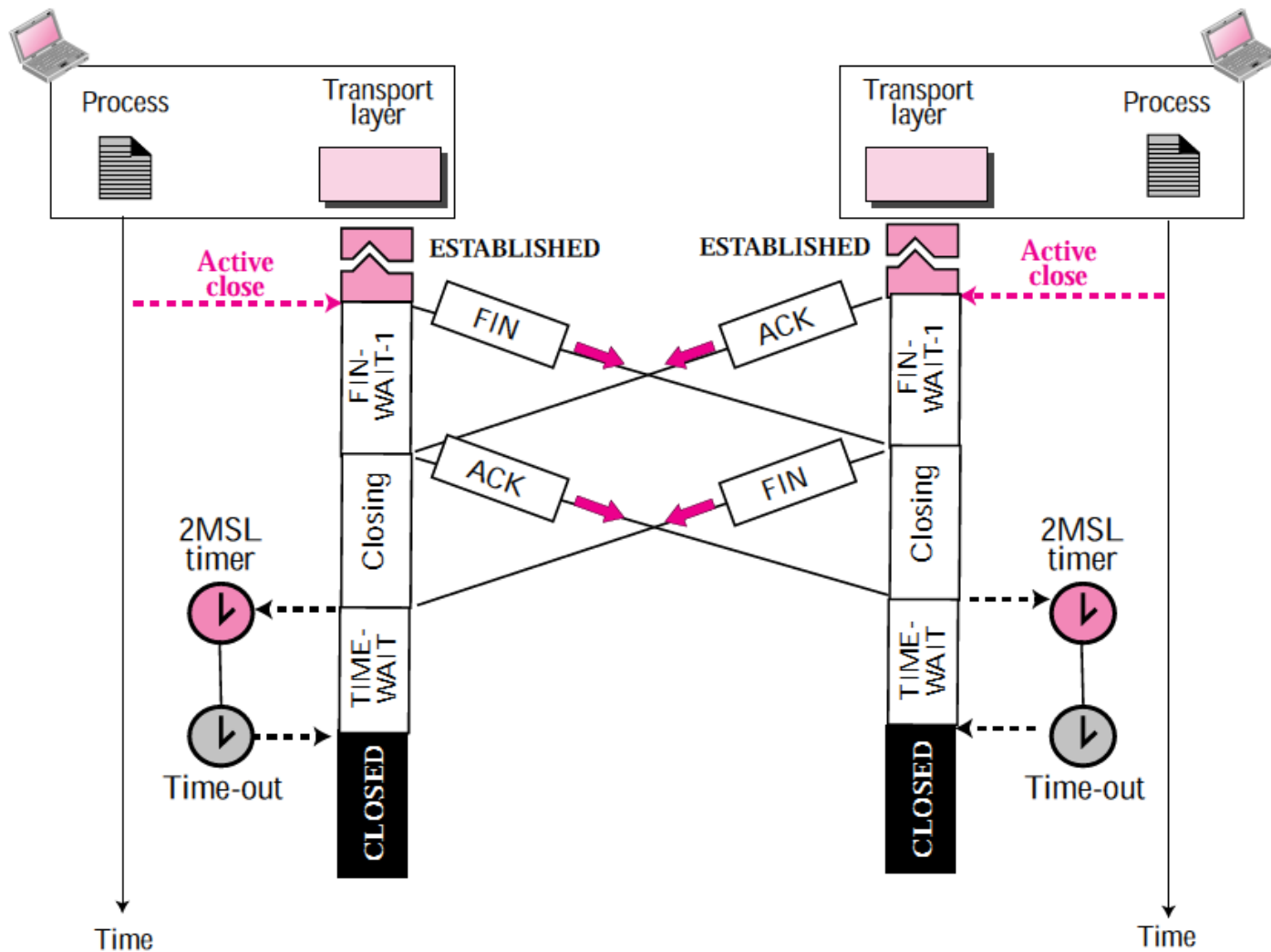


de la

# Exemple de Half-close



# Fermeture simultanée : segments FIN croisés



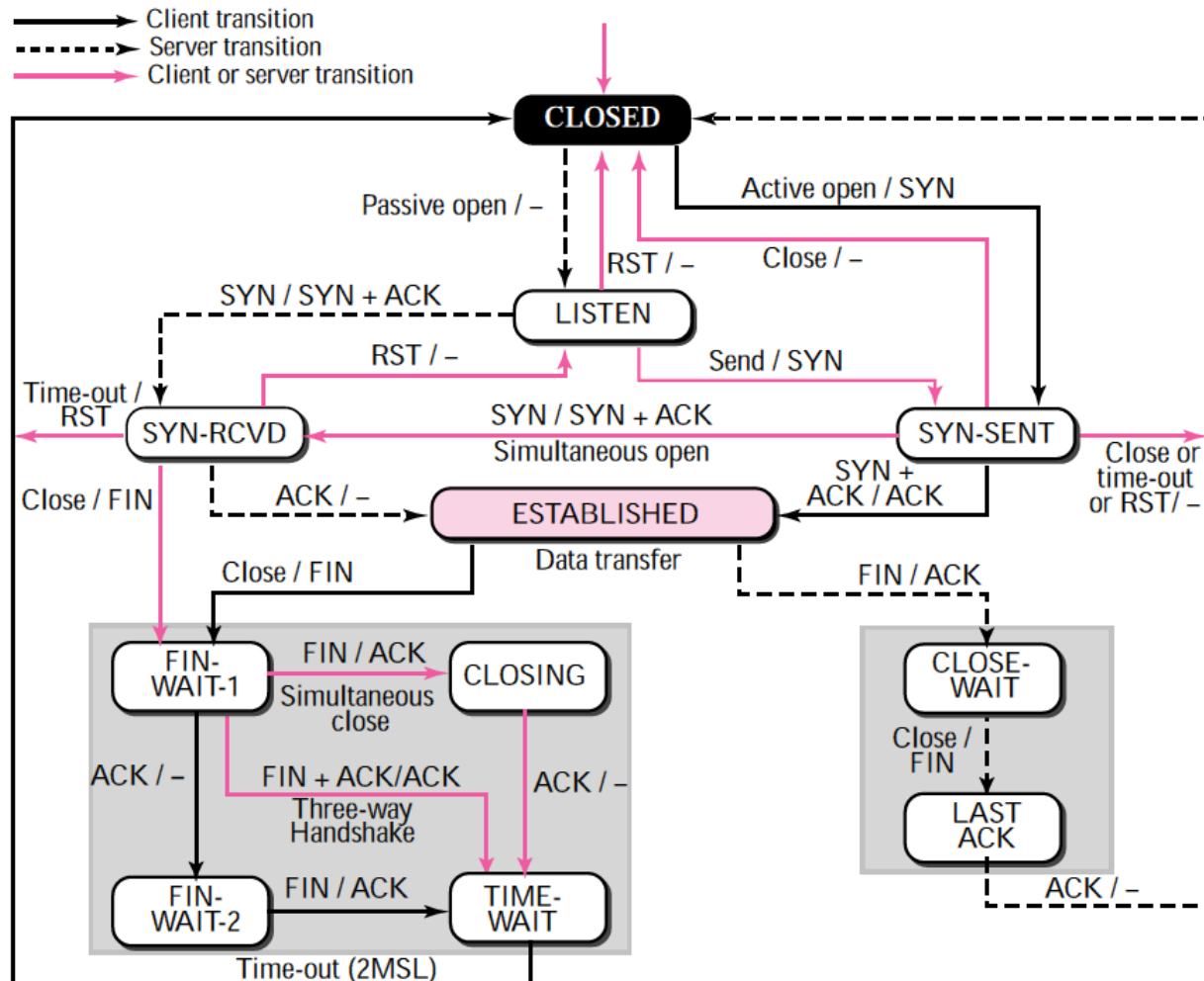
# Exercice

- Sur une connexion TCP, le ISN du client est de 2171. Le client ouvre une connexion, envoie un segment contenant 1000bytes de données et ferme la connexion. Quel est le numéro de séquence de chacun des segments suivants envoyés par le client
  - Segment SYN
  - Segment de données
  - Segment FIN

# Réinitialisation d'une connexion : RST

- Le flag RST activé permet :
  - De refuser une connexion
    - Exemple : refuser la connexion sur un numéro de port inactif
  - Annuler une connexion en cours
    - Exemple : Connexion dans un état incohérent
  - Terminer une connexion inactive
    - A cause d'un timeout

# Machine à état de TCP



# Machine à état de TCP

| <i>State</i>   | <i>Description</i>                                             |
|----------------|----------------------------------------------------------------|
| CLOSED         | No connection exists                                           |
| LI STEN        | Passive open received; waiting for SYN                         |
| SYN- SENT      | SYN sent; waiting for ACK                                      |
| SYN- RCVD      | SYN+ACK sent; waiting for ACK                                  |
| ESTABLI SHED   | Connection established; data transfer in progress              |
| FI N- WAI T- 1 | First FIN sent; waiting for ACK                                |
| FI N- WAI T- 2 | ACK to first FIN received; waiting for second FIN              |
| CLOSE- WAI T   | First FIN received, ACK sent; waiting for application to close |
| TI ME- WAI T   | Second FIN received, ACK sent; waiting for 2MSL time-out       |
| LAST- ACK      | Second FIN sent; waiting for ACK                               |
| CLOSI NG       | Both sides decided to close simultaneously                     |

# Transfert de données

- Le transfert des données est bidirectionnel
  - Du client vers le serveur et inversement
  - Les données d'un sens et les acquittements du sens opposé sont transportés dans le même segment
    - Le Piggybacking

The diagram illustrates the TCP connection establishment and termination process between a client and a server. The client side consists of a Client process and a Client transport layer. The server side consists of a Server transport layer and a Server process. The diagram is divided into two main sections: Connection Establishment and Connection Termination.

**Connection Establishment:**

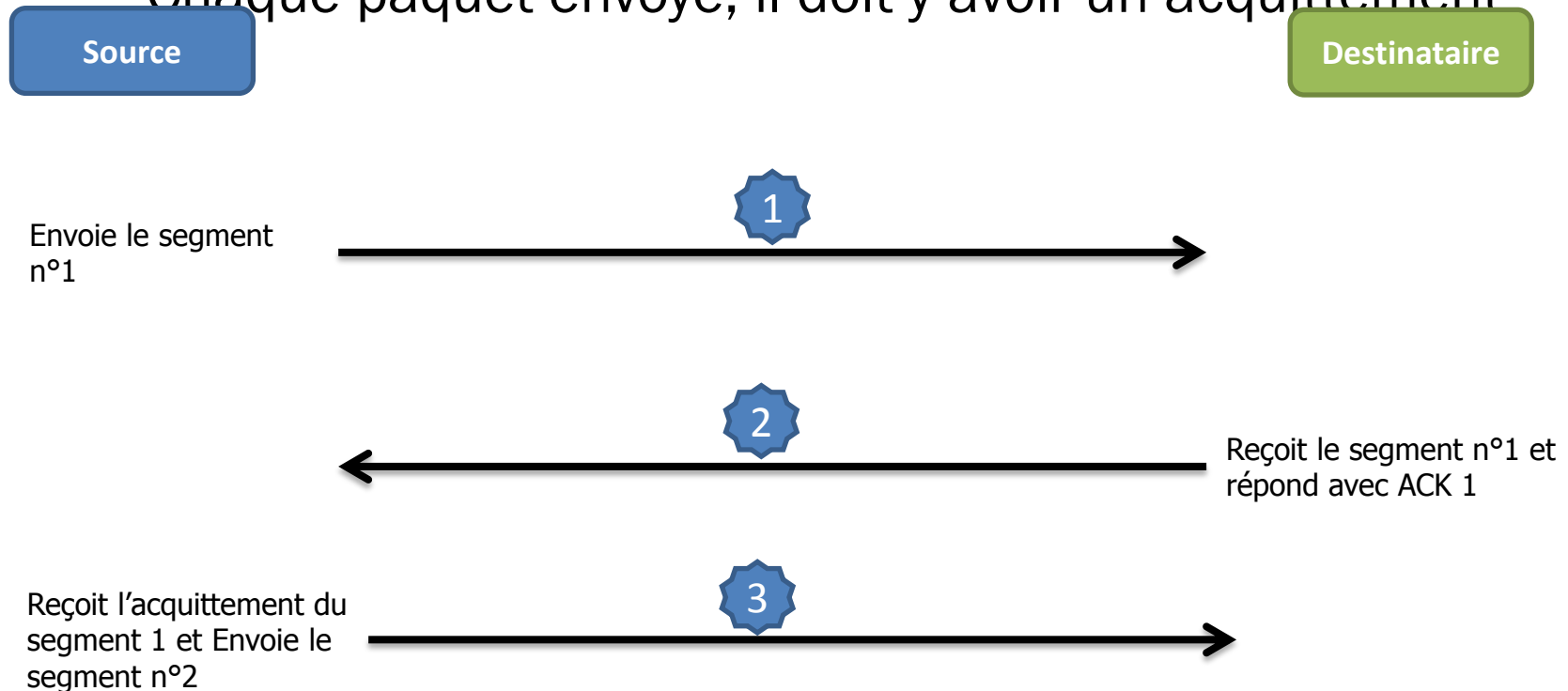
- Step 1:** The Client transport layer sends a request to the Client process. The request contains a segment with `seq: 8001`, `ack: 15001`, and `Data bytes: 8001-9000`. The segment has the `A` (ACK) flag set.
- Step 2:** The Client transport layer sends another request to the Client process. The request contains a segment with `seq: 9001`, `ack: 15001`, and `Data bytes: 9001-10000`. The segment has the `A` (ACK) flag set.
- Step 3:** The Server transport layer sends a request to the Server process. The request contains a segment with `seq: 15001`, `ack: 10001`, and `Data bytes: 15001-17000`. The segment has the `A` (ACK) flag set.
- Step 4:** The Client transport layer sends a request to the Client process. The request contains a segment with `seq: 10000`, `ack: 17001`, and `rwnd: 10000`. The segment has the `A` (ACK) flag set.

**Connection Termination:**

The diagram shows the termination of the connection, which typically involves a `FIN` (Finish) flag being set in a segment. However, the specific details of the termination sequence (e.g., `FIN` flag, `seq`, `ack`) are not explicitly labeled in the provided diagram.

# Gestion de la congestion : Fenêtres et Acquittements

- Principe intuitif des acquittements(naïf)
  - Chaque paquet envoyé, il doit y avoir un acquittement

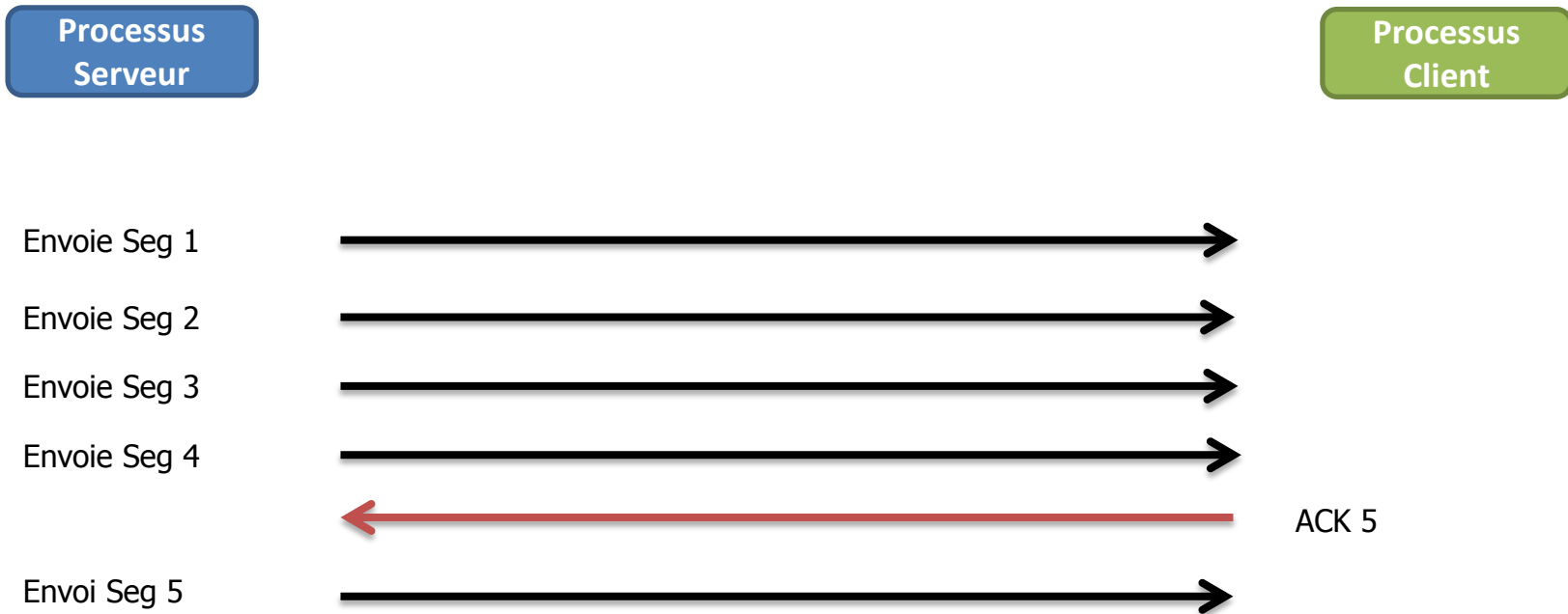


## Gestion de la congestion : Fenêtres et Acquittements

- Pour éviter d'avoir autant d'acquittement que de segments, TCP maintient des fenêtres
- Principe
  - Un certain nombre de segments sont envoyés sans attente d'acquittement : la fenêtre d'envoi(**window**)
  - Le récepteur doit acquitter chaque segment en indiquant le numéro de séquence du dernier paquet reçu sans erreur
  - Le récepteur indique la quantité de données qu'il peut recevoir sans manquer de ressources : fenêtre de réception

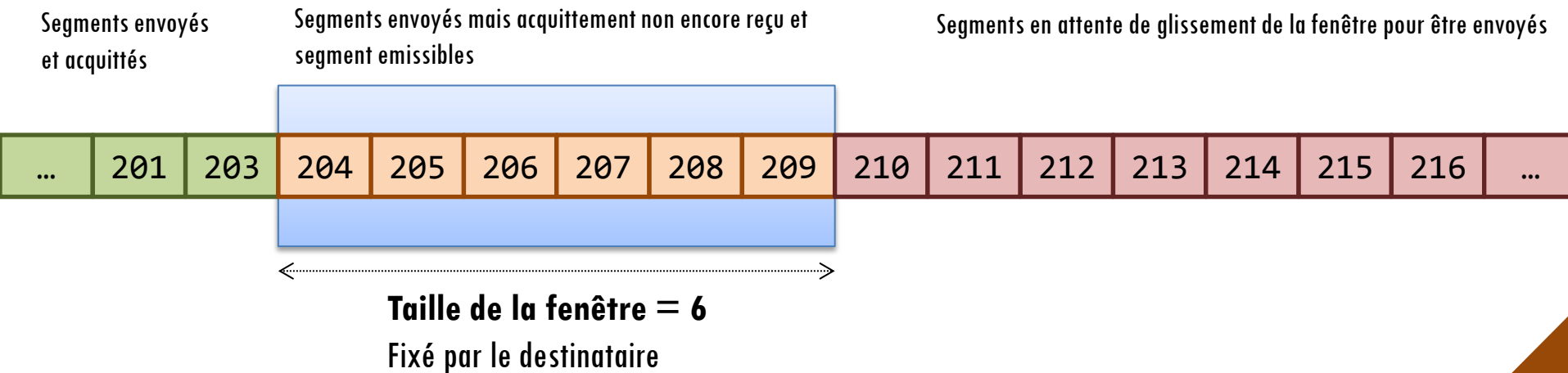
# Gestion de la congestion : Fenêtres et Acquittements

- Exemple de fenêtre de taille 5



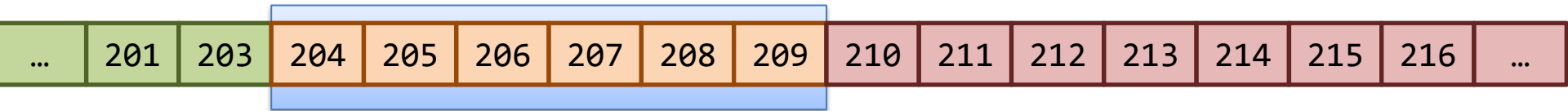
# Fenêtre d'envoi

- La fenêtre d'envoi gère l'état des octets envoyés en coulissant vers les paquets non traités
  - fenêtre glissante permettant d'optimiser la bande passante
  - permet également au destinataire de faire diminuer le débit de l'émetteur donc de gérer le contrôle de flux

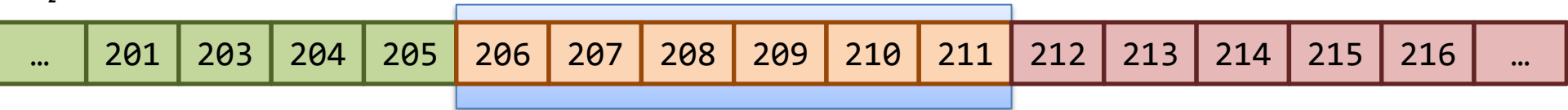


# Fenêtre d'envoi coulissante

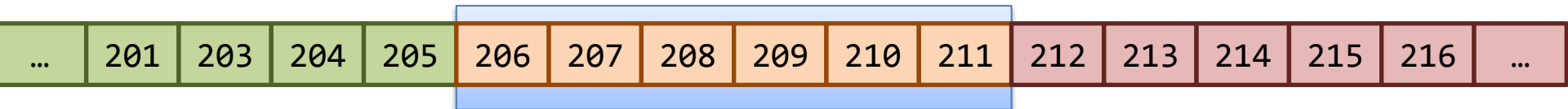
$t_1$



$t_2$  : réception de l'acquittement de 206



$t_3$  : réception de l'acquittement de 206 (segment perdu)



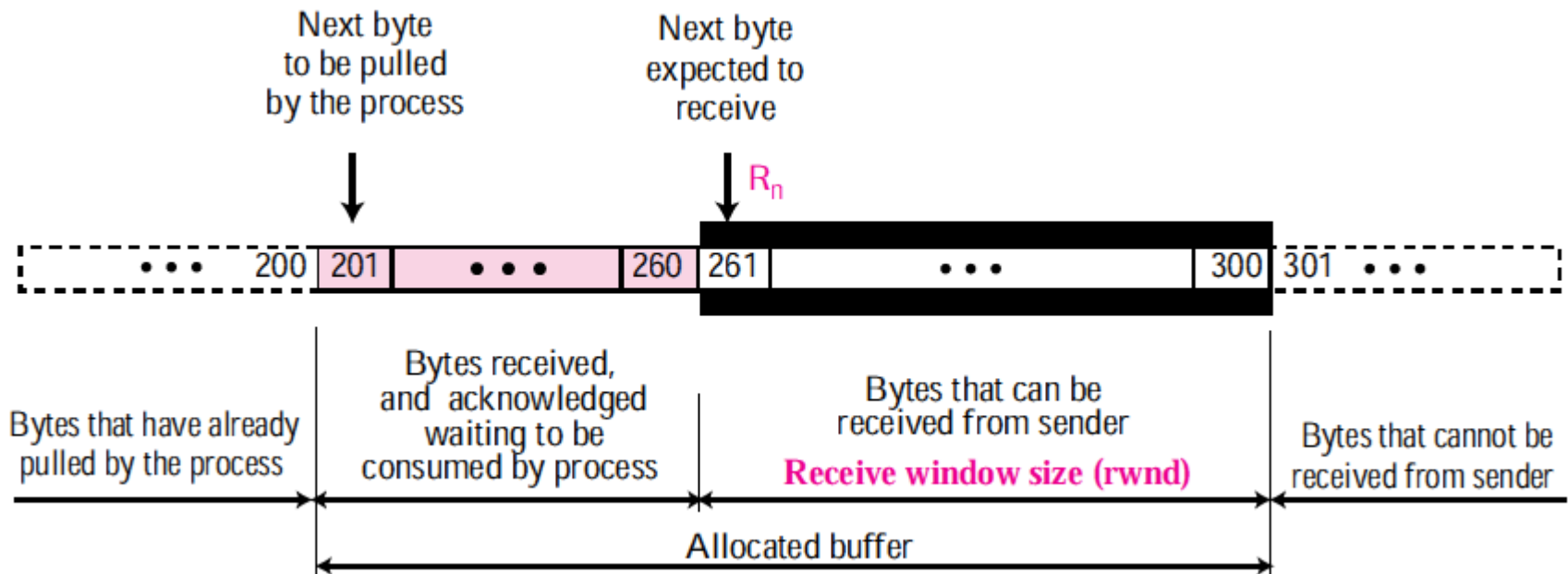
$t_4$  : réception de l'acquittement de 211



# Fenêtre de réception

- La fenêtre de réception (rwnd) dépend
  - de l'espace mémoire disponible au niveau du destinataire et de sa vitesse de traitement des données

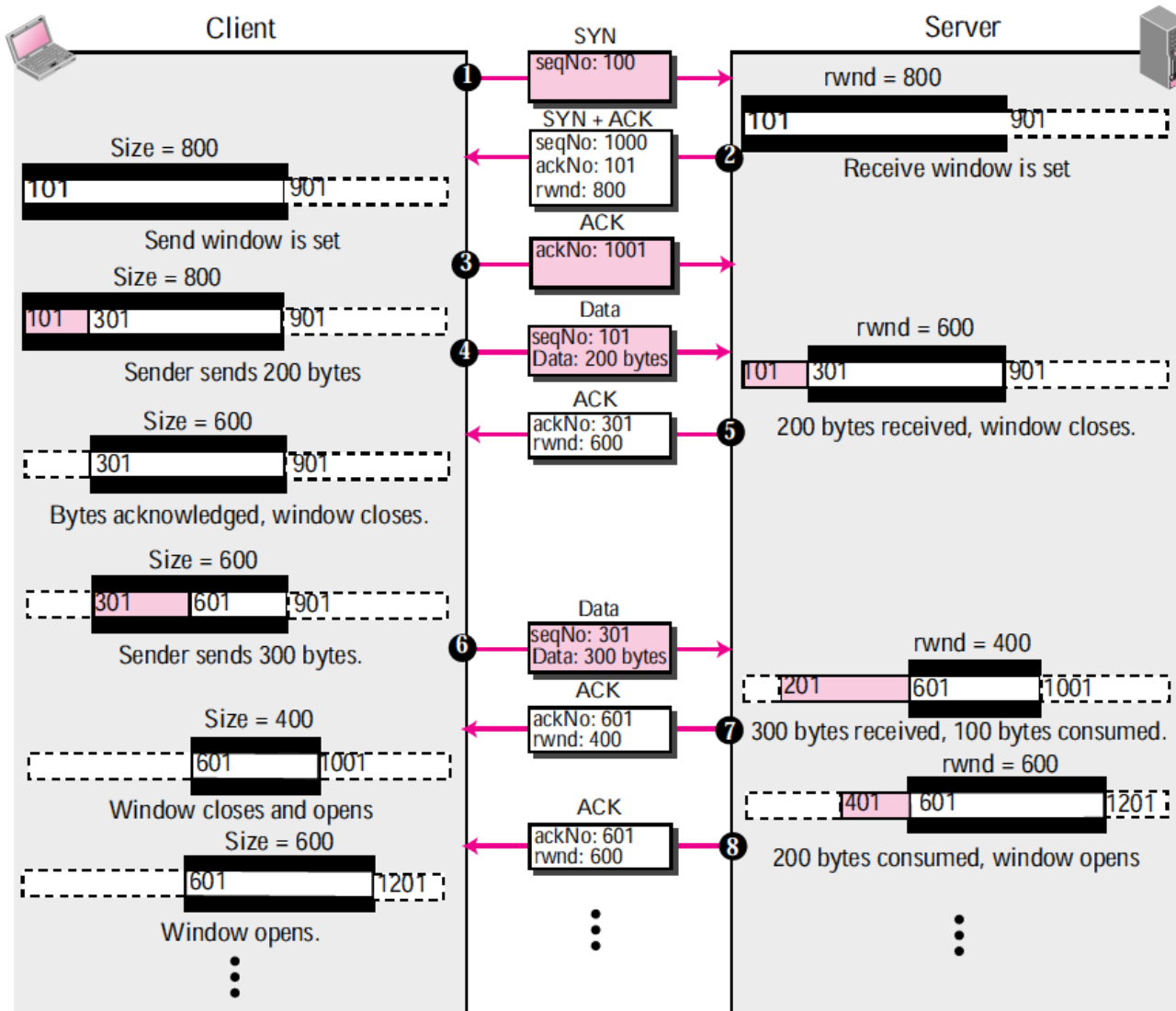
**rwnd = taille buffer - nombre d'octets en attente de traitement**



# Fenêtre de congestion

- La fenêtre de congestion est le minimum entre
  - la fenêtre de réception (rwnd)
  - la fenêtre acceptée par le réseau (cwnd)  
 $fenetreCongestion = minimum(rwnd, cwnd)$

# Fonctionnement des deux fenêtres

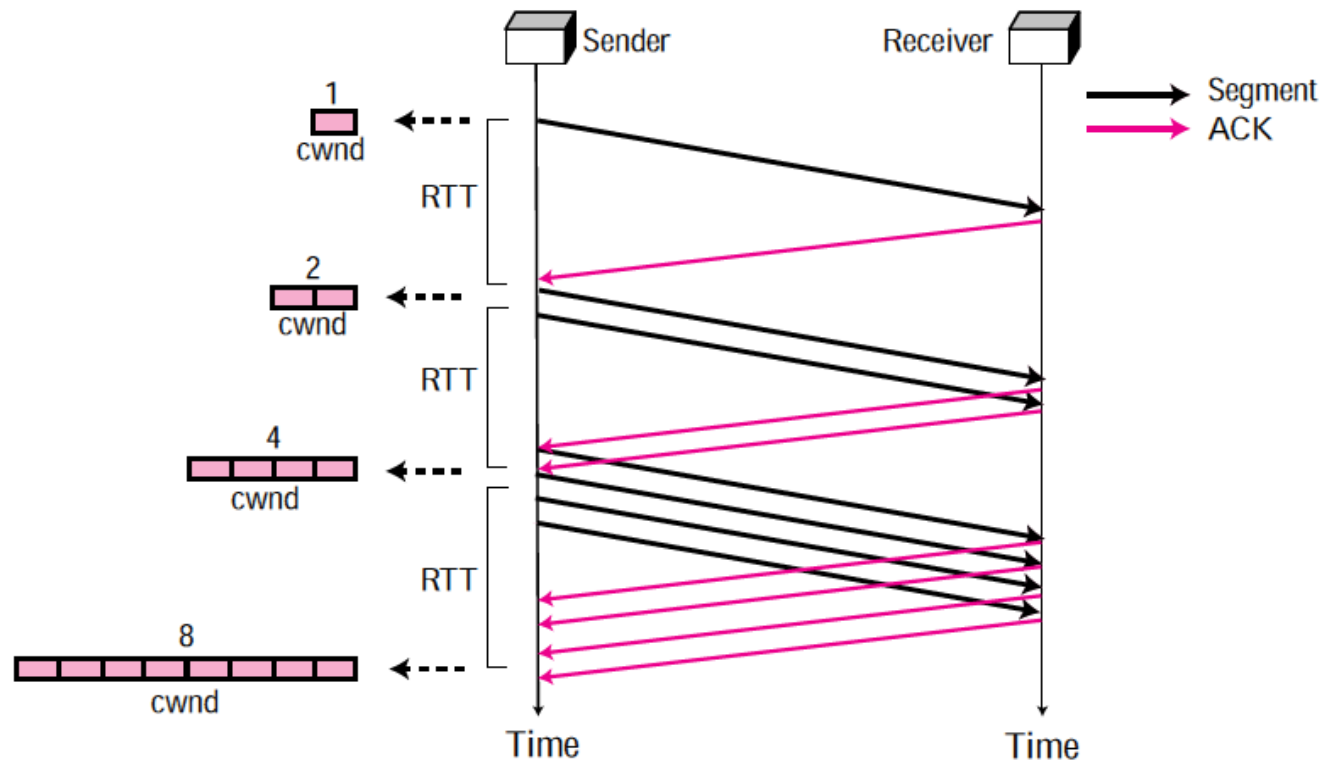


# Algorithmes de contrôle de congestion de TCP

- Les mécanismes de contrôle de congestion permettent à la source de ne pas provoquer des dépassements de capacités de la destination
- Plusieurs algorithmes ajoutés au fur et à mesure
  - Slow Start (départ lent)
  - Congestion Avoidance (évitement de congestion)
  - Fast retransmit (Retransmission rapide)
  - Fast Recovery (Reprise rapide)

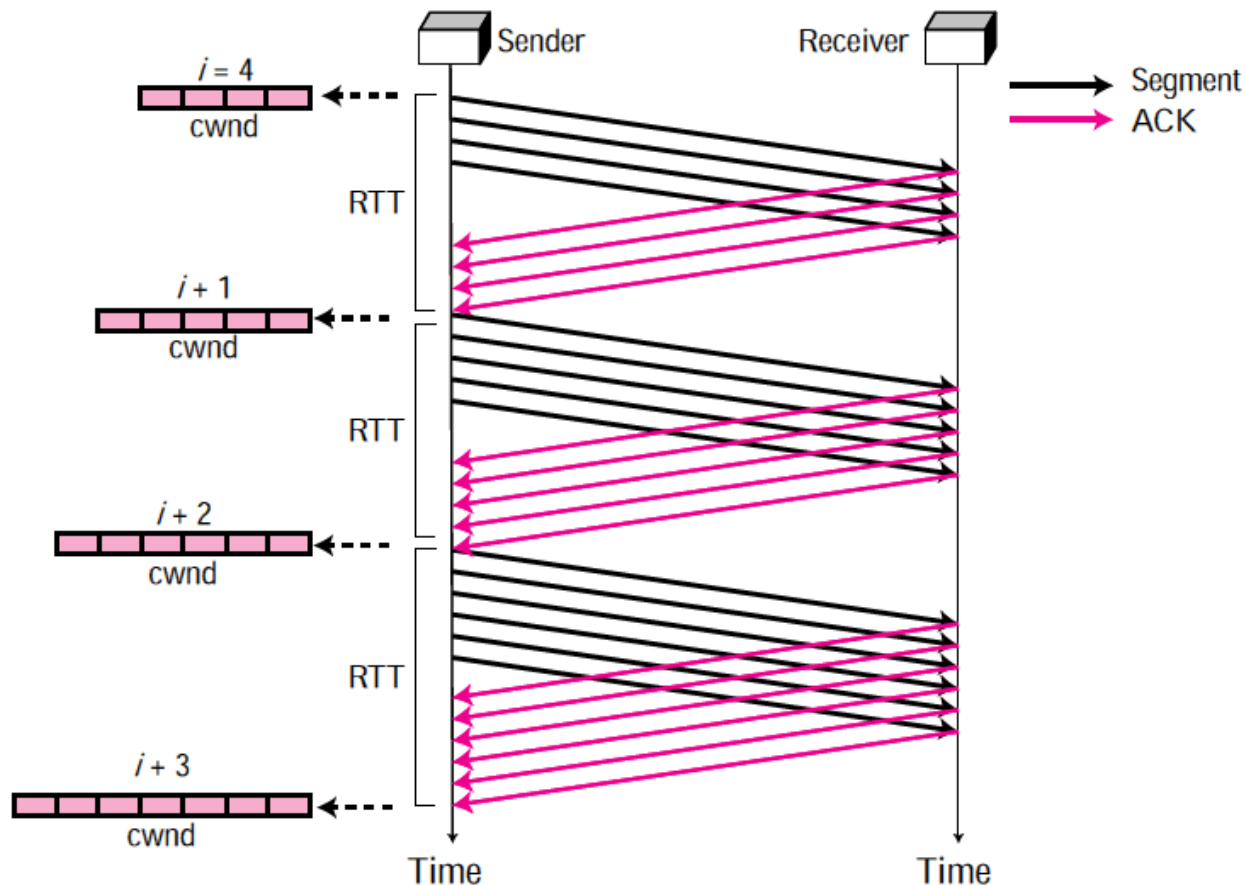
# Slow Start

- Fenêtre de congestion (**C**ongestion **W**i**N**Dow) commence par un segment
- Sa taille est augmentée progressivement de manière exponentielle



# Congestion avoidance : augmentation additive

- L'évitement de la congestion est un processus qui augmente la fenêtre de congestion **cwnd** de manière linéaire (addition)



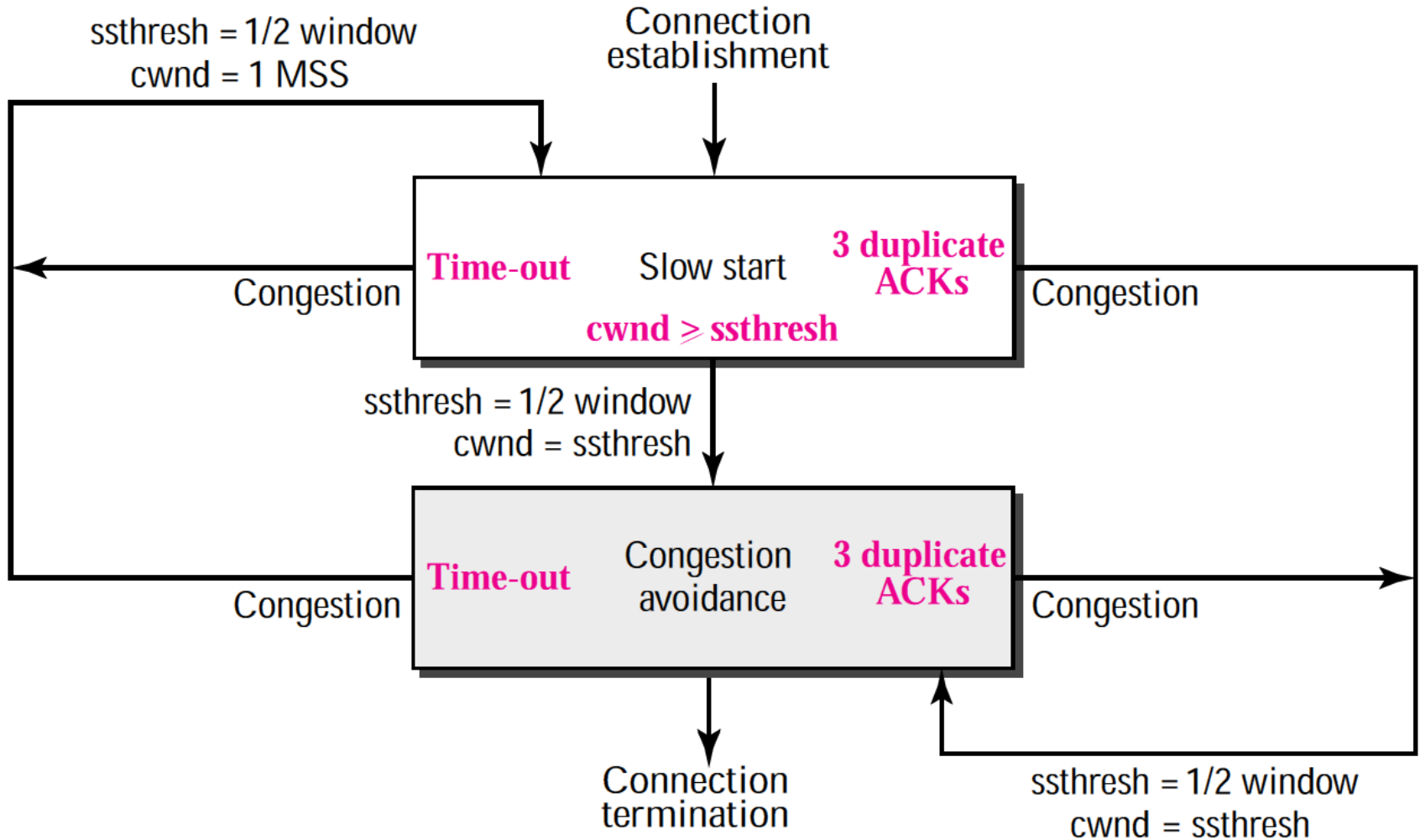
# Fonctionnement de congestion avoidance et Slow Start

- Hypothèse
  - Un segment perdu est le fait très probable de congestion avec une probabilité de 99%
- Indicateurs de congestion
  - Un *timeout*
  - Des acquittements dupliqués
- Variables en jeu
  - Cwnd
  - Slow Start Threshold *ssthresh*

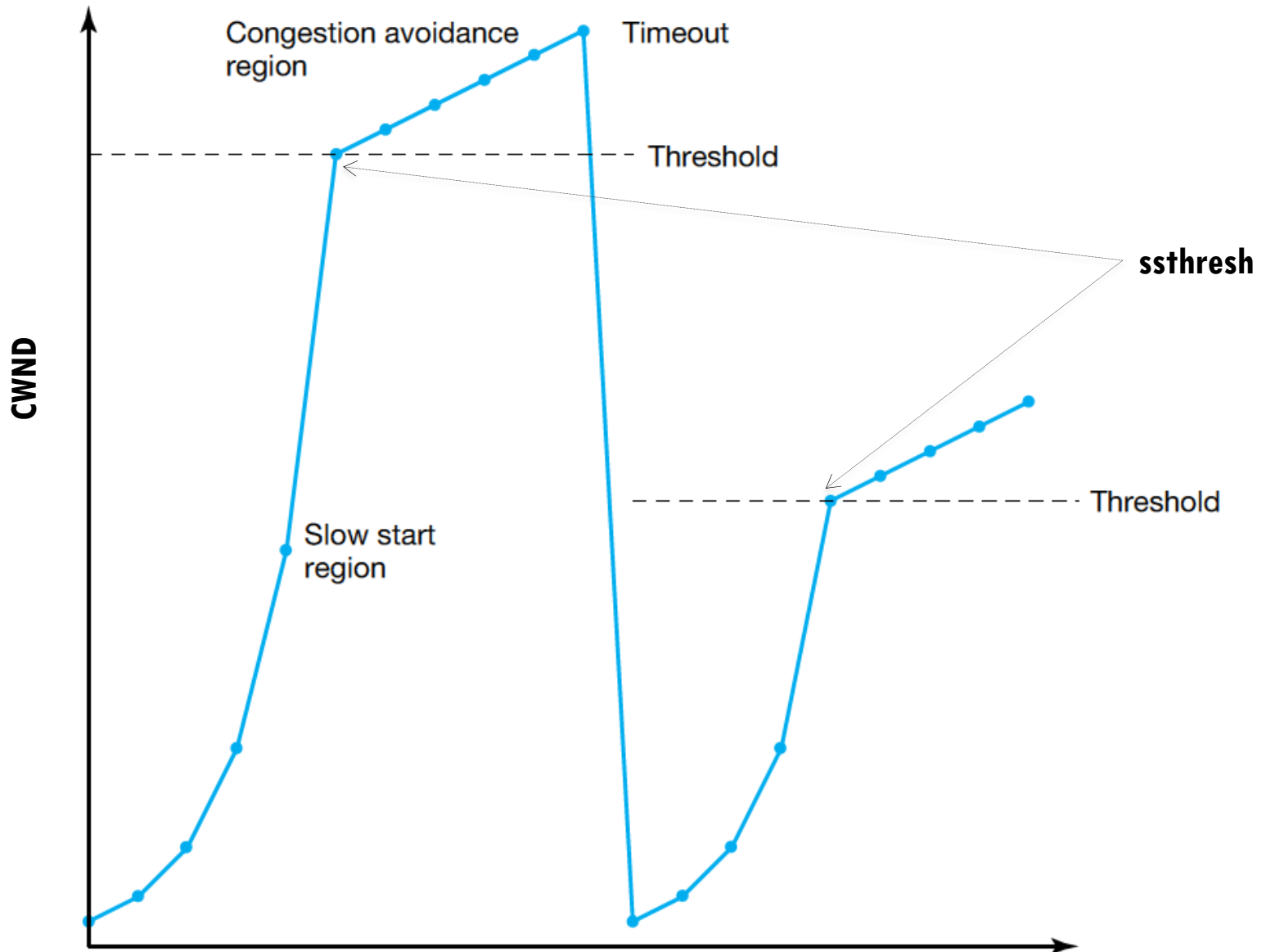
## Fonctionnement de *congestion avoidance* et *Slow Start*

- Principe
  - Lorsqu'il y a congestion  $ssthresh = cwnd / 2$
  - Si c'est un timeout  $cwnd = 1$
  - Si un nouvel acquittement est reçu TCP augmente  $cwnd$ 
    - Si  $cwnd < ssthresh \rightarrow$  Appliquer Slow Start
    - Sinon appliquer Congestion avoidance
- Le mécanisme congestion avoidance augmente la fenêtre par
$$Cwnd = segmentsize * segmentsize / cwnd$$
à chaque fois qu'un paquet est reçu

## Fonctionnement de congestion avoidance et Slow Start

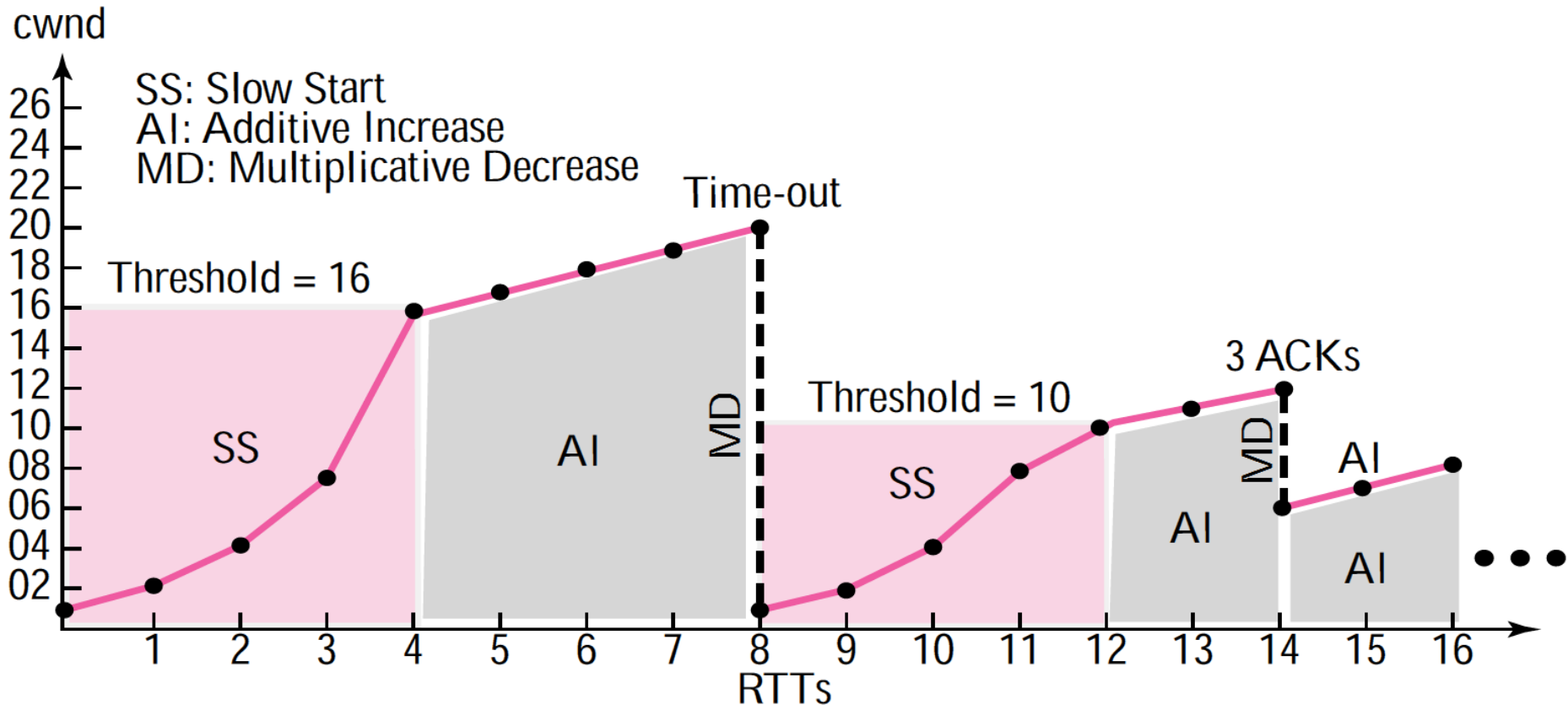


# Fonctionnement de congestion avoidance et Slow Start



# Fonctionnement de congestion avoidance et Slow Start

- Exemple d'évitement de congestion CO-SS



# Gestion des erreurs : acquittements

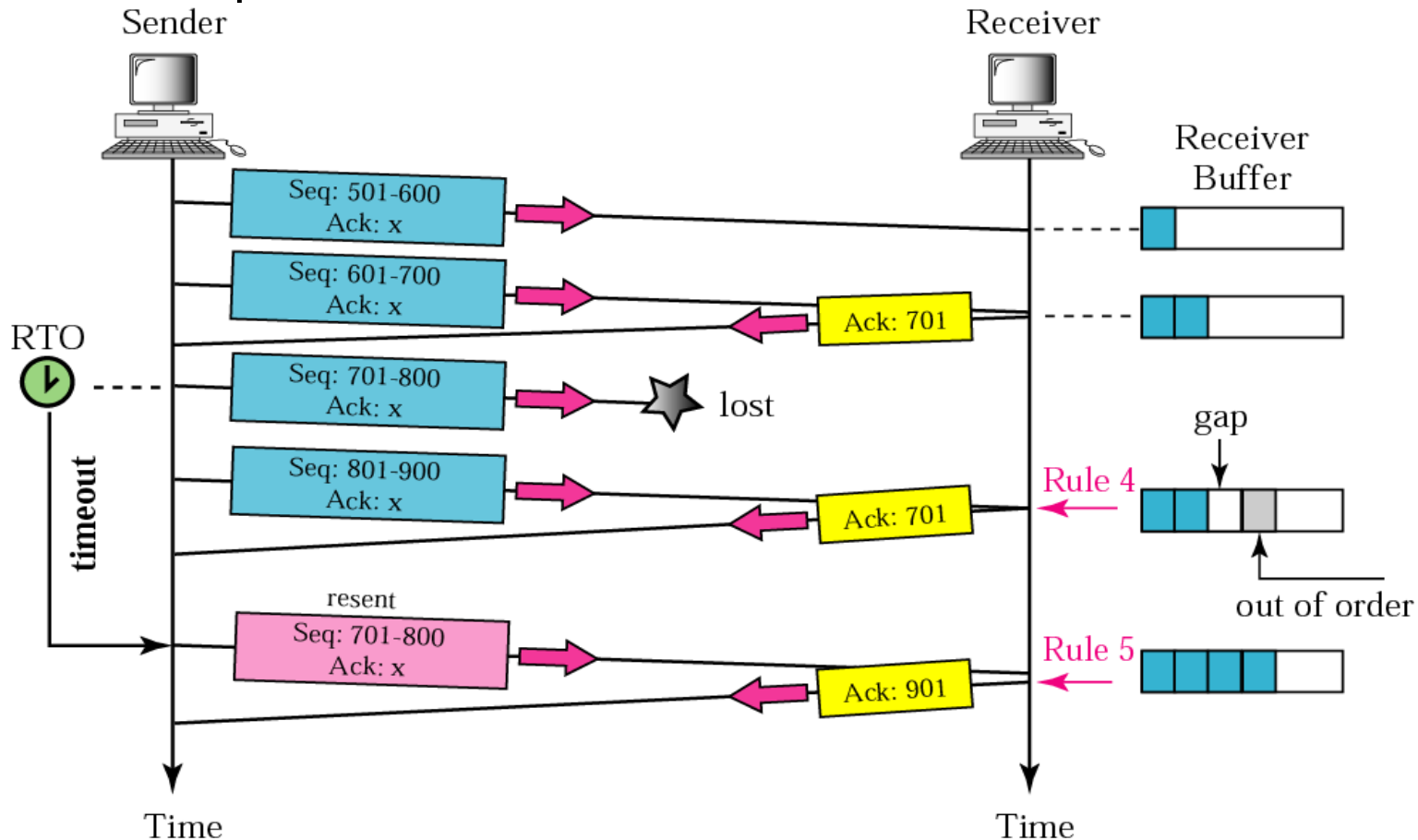
- 2 Types d'acquittements
- Acquittements cumulatifs
  - L'acquittement du segment  $i$  indique dans ce cas que tous les paquets avant ce segment ont été bien reçus
  - On l'appelle aussi l'acquittement cumulatif ou positif
- Acquittements sélectifs
  - Complète l'acquittement cumulatif
  - Permet de donner plus d'informations à la source comme :
    - Les paquets dupliqués
    - Les blocks arrivés dans le désordre

# Acquittement et retransmission

- Les paquets qui ne sont pas bien reçus sont retransmis par TCP
- Deux cas de retransmission
  - Retransmission après un RTO (Retransmission after RTO)
  - Trois acquittements dupliqués
- Les segments biens reçus après un segment perdu sont appelés des paquets *out-of-order*

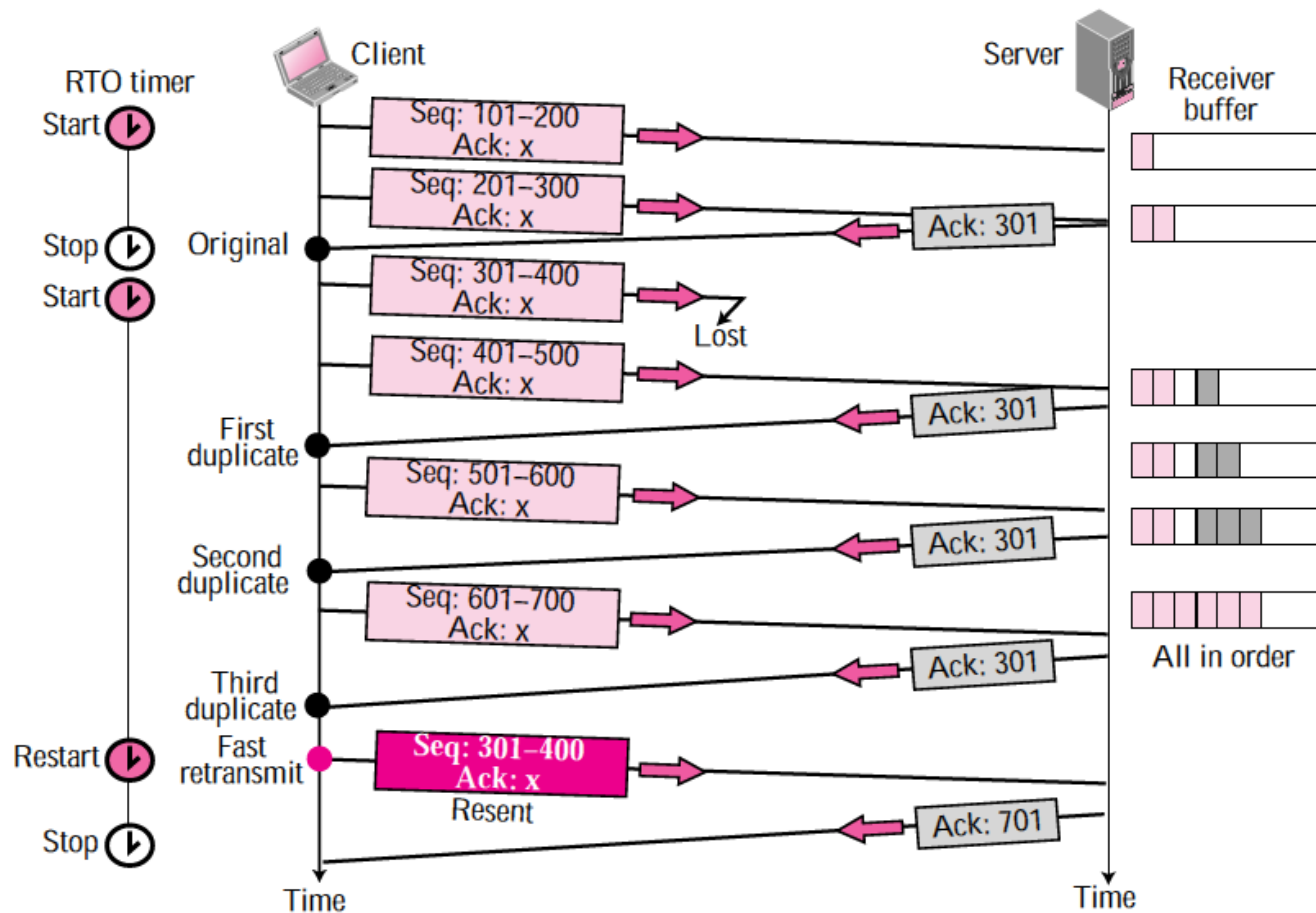
# Segments perdus

- Le dernier paquet bien reçu est acquitté pour indiquer à la source par où commencer la retransmission



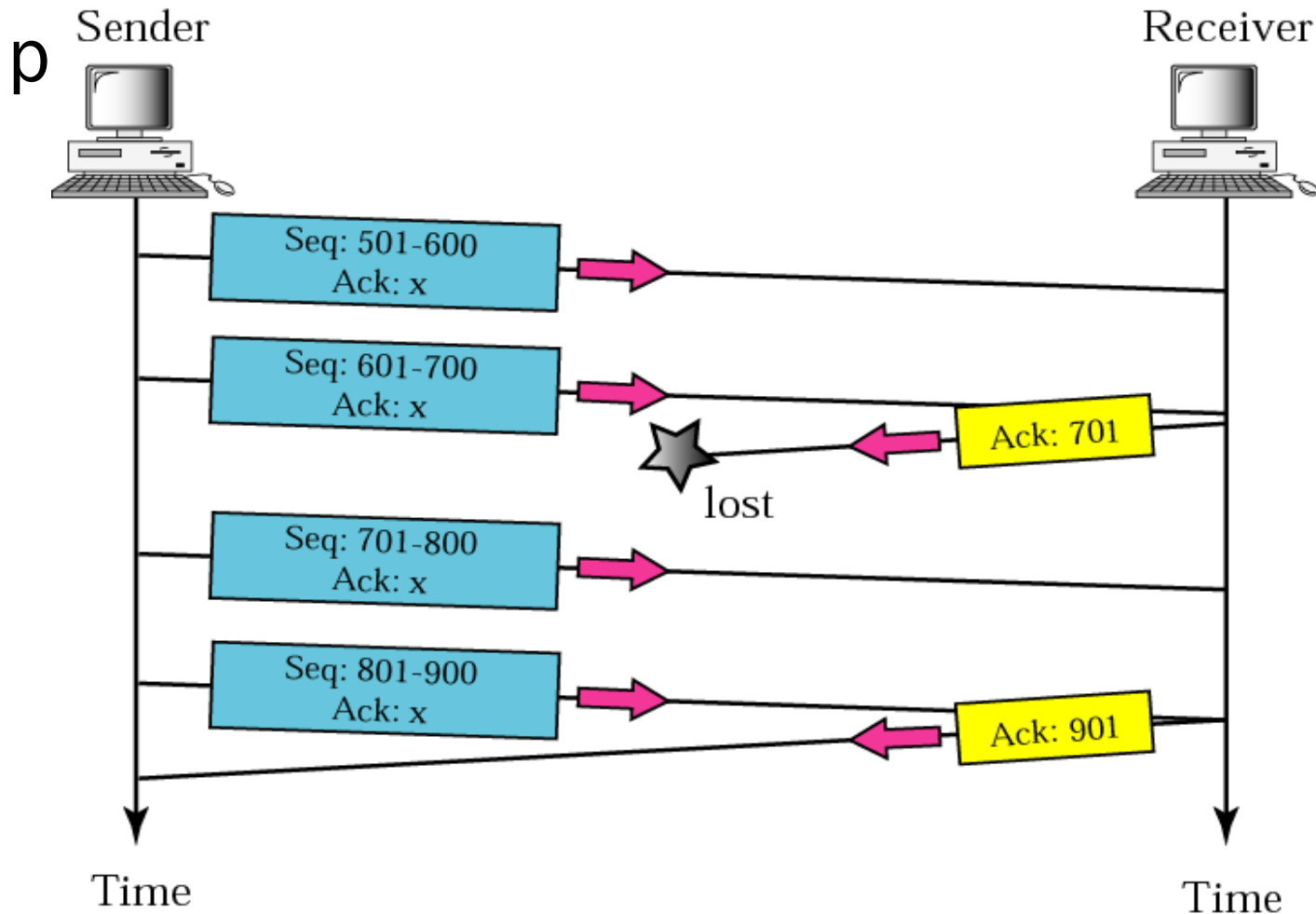
# Retransmission rapide(Fast Recovery)

- La retransmission rapide permet de ne pas attendre pour



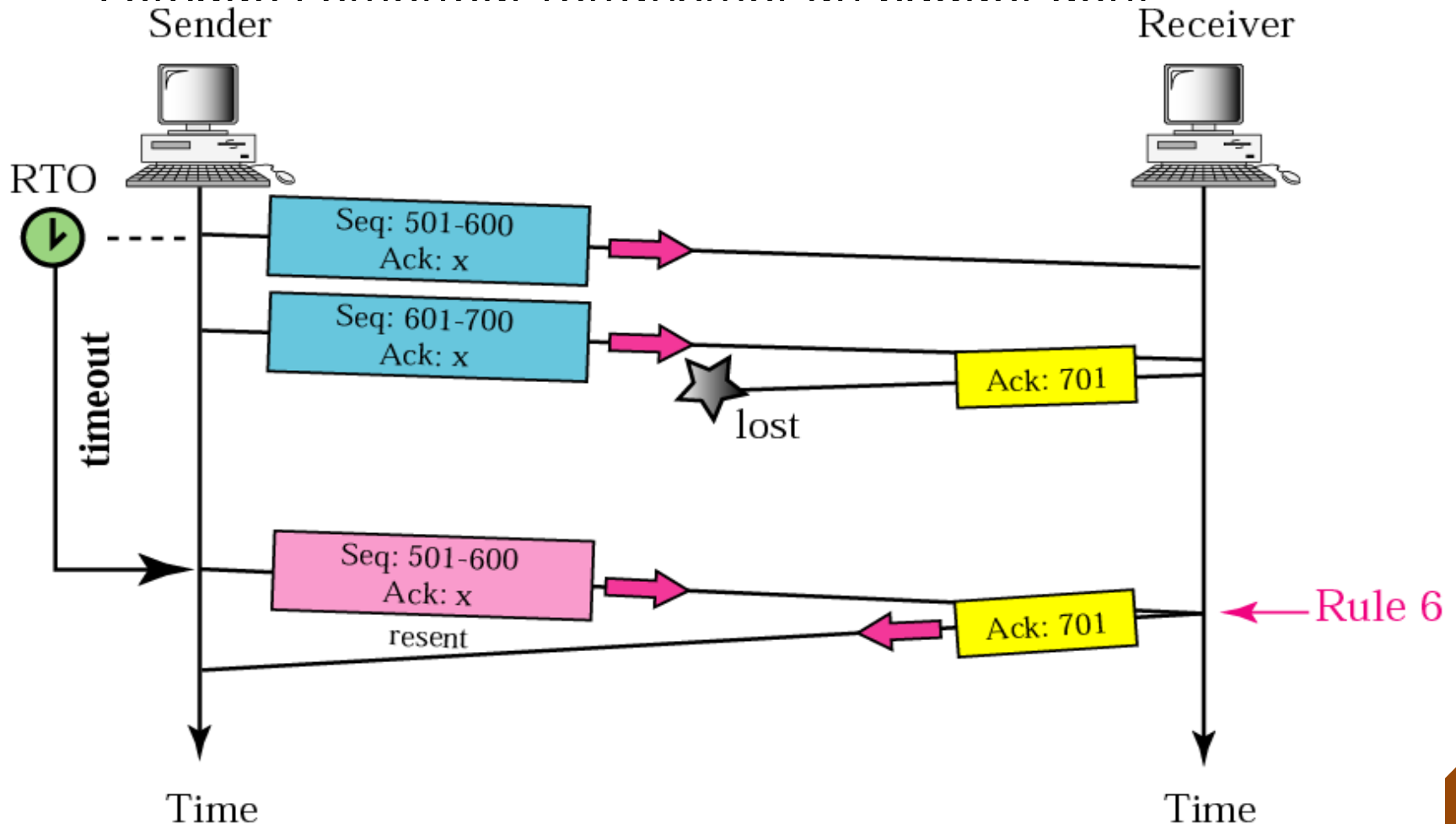
# Acquittement perdu : correction automatique

- Lorsque l'acquittement est perdu, le



## Acquittement perdu : Correction par l'envoi d'un nouvel ACK

- Lorsque l'émetteur retransmet un paquet dont



# Segment dupliqué

- Lorsqu'un segment est reçu avec un numéro de séquence d'un segment déjà reçu il est supprimé
  - Un ACK est envoyé avec le segment correct attendu
- Lorsqu'un segment arrive après avoir été retransmis
  - il est considéré comme segment dupliqué et supprimé par le destinataire

# Implémentation de TCP

